

ANDROID · TEST TASK · SOURCE CODE

شرح كود المشروع ملفاً ملفاً

FakeStore Auth — Token Handling App

لكل ملف: الكود الكامل منسّقاً مع أرقام الأسطر، ثم شرح مفصّل لما يفعله ولماذا كُتِب بهذه الطريقة.

MVVM

Retrofit + OkHttp

15

اختباراً ناجحاً

53

ملفاً مشروحاً

الجزء الخامس: أدوات الواجهة المشتركة

- ui/common/UiState.kt .28
- ui/common/ErrorMessage.kt .29
- ui/common/FlowExt.kt .30
- ui/common/Formatters.kt .31

الجزء السادس: الشاشات (View + ViewModel)

- res/values/strings.xml .32
- res/values/themes.xml .33
- res/layout/activity_main.xml .34
- res/navigation/nav_graph.xml .35
- MainActivity.kt .36
- ui/login/LoginViewModel.kt .37
- res/layout/fragment_login.xml .38
- ui/login/LoginFragment.kt .39
- res/layout/view_error.xml .40
- ui/products/ProductsViewModel.kt .41
- res/layout/item_product.xml .42
- ui/products/ProductAdapter.kt ★ .43
- res/layout/fragment_products.xml .44
- res/menu/menu_products.xml .45
- ui/products/ProductsFragment.kt .46
- ui/detail/ProductDetailViewModel.kt .47
- res/layout/fragment_product_detail.xml .48
- ui/detail/ProductDetailFragment.kt .49

الجزء السابع: الاختبارات (Unit Tests)

- test/.../TestDoubles.kt .50
- test/.../AuthManagerTest.kt .51
- test/.../AuthInterceptorTest.kt .52
- test/.../TokenAuthenticatorTest.kt .53

الجزء الثامن: الصورة الكاملة

- 54. رحلة طلب كاملة عبر الملفات
- 55. كيف تعرض المشروع للمدير (Demo)

0. قبل البدء: كيف تنقل المشروع إلى Android Studio

الجزء الأول: إعدادات المشروع (Gradle)

- gradle/libs.versions.toml .1
- settings.gradle.kts .2
- build.gradle.kts (الجزء) .3
- app/build.gradle.kts .4
- gradle.properties .5
- app/src/main/AndroidManifest.xml .6

الجزء الثاني: نماذج البيانات وال APIs

- data/model/Product.kt .7
- data/remote/ApiConfig.kt .8
- data/remote/dto/AuthDtos.kt .9
- data/remote/dto/ProductDto.kt .10
- data/remote/AuthApi.kt .11
- data/remote/ProductApi.kt .12

الجزء الثالث ★: منطق ال Token (قلب المهمة)

- data/auth/Clock.kt .13
- data/auth/AuthSession.kt .14
- data/auth/TokenStorage.kt .15
- data/auth/EncryptedTokenStorage.kt .16
- data/auth/AuthExceptions.kt .17
- data/auth/AuthEvent.kt .18
- data/auth/AuthHeaders.kt .19
- data/auth/AuthManager.kt ★★ .20
- data/auth/AuthInterceptor.kt .21
- data/auth/TokenAuthenticator.kt .22

الجزء الرابع: ال Repository وحقق الاعتماديات (Hilt)

- FakeStoreApp.kt .23
- data/repository/ProductRepository.kt .24
- di/Qualifiers.kt .25
- di/NetworkModule.kt ★ .26
- di/DataModule.kt .27

0. قبل البدء: كيف تنقل المشروع إلى Android Studio

الطريقة الأسهل (موصى بها)

1. حمّل الريبو من GitHub كملف ZIP، أو استخدم `git clone`.
2. افتح Android Studio، ثم اختر **Open** ← **File** وحدد مجلد المشروع.
3. انتظر انتهاء **Gradle Sync**، فهو يحمّل المكتبات تلقائياً.
4. اضغط **Run** ▶.

إذا أردت نسخ الملفات يدوياً إلى مشروع جديد

1. أنشئ مشروعاً جديداً من قالب **Empty Views Activity**.
2. اجعل اسم الحزمة **Package name** يساوي `com.example.fakestore`، واللغة **Kotlin**، و**Minimum SDK** يساوي 24.
3. انسخ الملفات بنفس المسارات المكتوبة في عنوان كل قسم من هذا الملف.
4. استبدل ملفات Gradle الموجودة بالملفات الواردة هنا، ثم اضغط **Sync Now**.

بنية المشروع

```
app/src/main/java/com/example/fakestore/
├── FakeStoreApp.kt           Application + Hilt
├── MainActivity.kt          the only Activity
├── data/
│   ├── model/Product.kt    domain model
│   ├── remote/             Retrofit APIs + DTOs
│   ├── auth/               ★ token logic (core of the task)
│   └── repository/         ProductRepository
├── di/                     Hilt modules
├── ui/
│   ├── common/             UiState, errors, helpers
│   ├── login/              Login screen
│   ├── products/           list (RecyclerView)
│   └── detail/              product details
├── app/src/main/res/        layouts, navigation, menu, strings
└── app/src/test/.../data/auth/ unit tests for the token logic
```

ترتيب القراءة المقترح

اقرأ الأقسام بالترتيب، فكل قسم يعتمد على ما قبله:

الجزء	المحتوى
الأول	إعدادات Gradle وال Manifest
الثاني	نماذج البيانات وال APIs
الثالث ★	منطق ال Token، وهو قلب المهمة
الرابع	ال Repository وحقن الاعتماديات (Hilt)
الخامس	أدوات الواجهة المشتركة
السادس	الشاشات
السابع	الاختبارات

الجزء	المحتوى
الثامن	الرحلة الكاملة للطلب، وكيف تعرض المشروع للمدير

ملاحظة: أرقام الأسطر الظاهرة بجانب الكود تساعدك على متابعة الشرح.

الجزء الأول: إعدادات المشروع (Gradle)

Gradle هو نظام البناء في Android. يقرأ هذه الملفات ليعرف:

- ما المكتبات التي يحتملها.
- كيف يبني التطبيق.

يقابل ملف `pubspec.yaml` في Flutter.

1. `gradle/libs.versions.toml`

الدور: كتالوج الإصدارات (Version Catalog). كل أسماء المكتبات وإصداراتها في مكان واحد.

gradle/libs.versions.toml

51 lines

```
1 [versions]
2 agp = "8.5.2"
3 kotlin = "2.0.20"
4 ksp = "2.0.20-1.0.25"
5 hilt = "2.52"
6 coreKtx = "1.13.1"
7 appcompat = "1.7.0"
8 material = "1.12.0"
9 constraintlayout = "2.1.4"
10 recyclerview = "1.3.2"
11 fragmentKtx = "1.8.3"
12 lifecycle = "2.8.5"
13 navigation = "2.8.0"
14 securityCrypto = "1.1.0-alpha06"
15 retrofit = "2.11.0"
16 okhttp = "4.12.0"
17 coil = "2.7.0"
18 coroutines = "1.8.1"
19 junit = "4.13.2"
20
21 [libraries]
22 androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
23 androidx-appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref = "appcompat" }
24 material = { group = "com.google.android.material", name = "material", version.ref = "material" }
25 androidx-constraintlayout = { group = "androidx.constraintlayout", name = "constraintlayout",
version.ref = "constraintlayout" }
26 androidx-recyclerview = { group = "androidx.recyclerview", name = "recyclerview", version.ref =
"recyclerview" }
27 androidx-fragment-ktx = { group = "androidx.fragment", name = "fragment-ktx", version.ref =
"fragmentKtx" }
28 androidx-lifecycle-viewmodel-ktx = { group = "androidx.lifecycle", name = "lifecycle-viewmodel-ktx",
version.ref = "lifecycle" }
29 androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx",
version.ref = "lifecycle" }
30 androidx-navigation-fragment-ktx = { group = "androidx.navigation", name = "navigation-fragment-ktx",
version.ref = "navigation" }
31 androidx-navigation-ui-ktx = { group = "androidx.navigation", name = "navigation-ui-ktx", version.ref =
"navigation" }
32 androidx-security-crypto = { group = "androidx.security", name = "security-crypto", version.ref =
"securityCrypto" }
33
34 hilt-android = { group = "com.google.dagger", name = "hilt-android", version.ref = "hilt" }
35 hilt-compiler = { group = "com.google.dagger", name = "hilt-android-compiler", version.ref = "hilt" }
```

```

36
37 retrofit = { group = "com.squareup.retrofit2", name = "retrofit", version.ref = "retrofit" }
38 retrofit-converter-gson = { group = "com.squareup.retrofit2", name = "converter-gson", version.ref =
"retrofit" }
39 okhttp = { group = "com.squareup.okhttp3", name = "okhttp", version.ref = "okhttp" }
40 okhttp-logging = { group = "com.squareup.okhttp3", name = "logging-interceptor", version.ref = "okhttp"
}
41 okhttp-mockwebserver = { group = "com.squareup.okhttp3", name = "mockwebserver", version.ref = "okhttp"
}
42 coil = { group = "io.coil-kt", name = "coil", version.ref = "coil" }
43 kotlinx-coroutines-android = { group = "org.jetbrains.kotlinx", name = "kotlinx-coroutines-android",
version.ref = "coroutines" }
44 kotlinx-coroutines-test = { group = "org.jetbrains.kotlinx", name = "kotlinx-coroutines-test",
version.ref = "coroutines" }
45 junit = { group = "junit", name = "junit", version.ref = "junit" }
46
47 [plugins]
48 android-application = { id = "com.android.application", version.ref = "app" }
49 kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
50 ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
51 hilt = { id = "com.google.dagger.hilt.android", version.ref = "hilt" }

```

الشرح

الملف مقسوم إلى ثلاثة أقسام:

القسم	ماذا يحتوي
[versions]	أرقام الإصدارات فقط، مثل <code>retrofit = "2.11.0"</code>
[libraries]	المكتبات. كل مكتبة لها <code>group</code> و <code>name</code> ، وتشير إلى رقم إصدارها عبر <code>version.ref</code>
[plugins]	الإضافات (Plugins) التي تغيّر طريقة البناء نفسها

لماذا هذا الأسلوب؟ عند تحديث مكتبة تغيّر رقماً واحداً فقط. وفي ملفات Gradle تكتب `libs.retrofit` بدل النص الطويل `com.squareup.retrofit2:retrofit:2.11.0`.

أهم المكتبات ولماذا اخترناها:

المكتبة	لماذا
<code>converter-gson</code> + <code>retrofit</code>	تعريف طلبات الـ API وتحويل JSON إلى كائنات Kotlin. مطلوبة في المهمة
<code>logging-interceptor</code> + <code>okhttp</code>	المحرك الذي ينقذ الطلبات، وفيه الـ Interceptors. مطلوبة في المهمة
<code>security-crypto</code>	التخزين المشفّر للـ token
<code>hilt</code>	حقن الاعتماديات (Dependency Injection)
<code>navigation-ui</code> - / <code>navigation-fragment-ktx</code> <code>ktx</code>	التنقل بين الشاشات
<code>recyclerview</code>	القائمة. مطلوبة في المهمة
<code>coil</code>	تحميل صور المنتجات من الإنترنت
<code>mockwebserver</code>	سيرفر وهمي محلي للاختبارات

الدور: يحدد أين يبحث Gradle عن المكتبات، وما الوحدات (modules) الموجودة في المشروع.

settings.gradle.kts

18 lines

```

1 pluginManagement {
2     repositories {
3         google()
4         mavenCentral()
5         gradlePluginPortal()
6     }
7 }
8
9 dependencyResolutionManagement {
10    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
11    repositories {
12        google()
13        mavenCentral()
14    }
15 }
16
17 rootProject.name = "FakeStoreAuth"
18 include(":app")

```

الشرح

- `pluginManagement.repositories`: المستودعات التي تُحمَّل منها الإضافات (Plugins).
- `dependencyResolutionManagement`: المستودعات التي تُحمَّل منها المكتبات:
 - `google()`: مكتبات Android.
 - `mavenCentral()`: باقي المكتبات مثل Retrofit و OkHttp.
- `FAIL_ON_PROJECT_REPOS`: يمنع أي وحدة من إضافة مستودعات خاصة بها، فتبقى كل المصادر في مكان واحد.
- `include(":app")`: المشروع فيه وحدة واحدة اسمها `app`.

3. build.gradle.kts (الجزر)

الدور: يعلن الإضافات على مستوى المشروع دون أن يطبقها.

build.gradle.kts

6 lines

```

1 plugins {
2     alias(libs.plugins.android.application) apply false
3     alias(libs.plugins.kotlin.android) apply false
4     alias(libs.plugins.ksp) apply false
5     alias(libs.plugins.hilt) apply false
6 }

```

الشرح

`apply false` تعني: "حمّل هذه الإضافة لكن لا تطبقها هنا". كل وحدة تطبق ما تحتاجه منها، وفي مشروعنا الوحدة الوحيدة هي `app`.

الدور: ملف بناء التطبيق نفسه، وهو أهم ملفات Gradle.

app/build.gradle.kts

83 lines

```
1 plugins {
2     alias(libs.plugins.android.application)
3     alias(libs.plugins.kotlin.android)
4     alias(libs.plugins.ksp)
5     alias(libs.plugins.hilt)
6 }
7
8 android {
9     namespace = "com.example.fakestore"
10    compileSdk = 34
11
12    defaultConfig {
13        applicationId = "com.example.fakestore"
14        minSdk = 24
15        targetSdk = 34
16        versionCode = 1
17        versionName = "1.0"
18    }
19
20    buildTypes {
21        release {
22            isMinifyEnabled = false
23            proguardFiles(
24                getDefaultProguardFile("proguard-android-optimize.txt"),
25                "proguard-rules.pro",
26            )
27        }
28    }
29
30    compileOptions {
31        sourceCompatibility = JavaVersion.VERSION_17
32        targetCompatibility = JavaVersion.VERSION_17
33    }
34
35    kotlinOptions {
36        jvmTarget = "17"
37    }
38
39    buildFeatures {
40        viewBinding = true
41        buildConfig = true
42    }
43 }
44
45 dependencies {
46     // AndroidX + Material UI
47     implementation(libs.androidx.core.ktx)
48     implementation(libs.androidx.appcompat)
49     implementation(libs.material)
50     implementation(libs.androidx.constraintlayout)
51     implementation(libs.androidx.recyclerview)
52     implementation(libs.androidx.fragment.ktx)
53
54     // MVVM
55     implementation(libs.androidx.lifecycle.viewmodel.ktx)
56     implementation(libs.androidx.lifecycle.runtime.ktx)
57     implementation(libs.kotlinx.coroutines.android)
```

```

58
59 // Navigation between fragments
60 implementation(libs.androidx.navigation.fragment.ktx)
61 implementation(libs.androidx.navigation.ui.ktx)
62
63 // Secure token storage
64 implementation(libs.androidx.security.crypto)
65
66 // Networking
67 implementation(libs.retrofit)
68 implementation(libs.retrofit.converter.gson)
69 implementation(libs.okhttp)
70 implementation(libs.okhttp.logging)
71
72 // Image loading
73 implementation(libs.coil)
74
75 // Dependency injection
76 implementation(libs.hilt.android)
77 ksp(libs.hilt.compiler)
78
79 // Unit tests (JVM)
80 testImplementation(libs.junit)
81 testImplementation(libs.kotlinx.coroutines.test)
82 testImplementation(libs.okhttp.mockwebserver)
83 }

```

الشرح

الإضافات (plugins):

الإضافة	ماذا تفعل
android.application	تجعل الوحدة تطبيق Android قابلاً للتثبيت
kotlin.android	دعم لغة Kotlin
ksp	Kotlin Symbol Processing : يولّد كوداً وقت البناء، ويحتاجه Hilt
hilt	يفعل Hilt

إعدادات { android }:

الإعداد	المعنى
applicationId / namespace	المعرّف الفريد للتطبيق
compileSdk = 34	تُترجم الكود مقابل Android 14
minSdk = 24	أقل نسخة مدعومة: Android 7.0
targetSdk = 34	النسخة التي اختُبر عليها التطبيق
kotlinOptions / compileOptions	استخدام Java 17

الميزات (buildFeatures):

- `viewBinding = true`: يولّد لكل ملف XML كلاساً يحمل مراجع عناصره، مثلاً `fragment_login.xml` ← `FragmentLoginBinding`. هذا يغني عن `findViewById` ويمنع أخطاء الأنواع والقيم الفارغة.

- `buildConfig = true`: يُولد كلاس `BuildConfig`. نستخدم منه `BuildConfig.DEBUG` لتفعيل سجل الشبكة في نسخة التطوير فقط.

أنواع الاعتماديات (dependencies):

النوع	متى يُستخدم
<code>implementation</code>	مكتبة يحتاجها التطبيق
<code>ksp(...)</code>	مولّد كود يعمل وقت البناء فقط
<code>testImplementation</code>	مكتبة للاختبارات فقط، ولا تدخل في التطبيق النهائي

5. `gradle.properties`

الدور: إعدادات عامة لـ Gradle.

gradle.properties4 lines

```
1 org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
2 android.useAndroidX=true
3 android.nonTransitiveRClass=true
4 kotlin.code.style=official
```

الشرح

الإعداد	المعنى
<code>org.gradle.jvmargs</code>	ذاكرة Gradle أثناء البناء
<code>android.useAndroidX</code>	استخدام مكتبات AndroidX الحديثة
<code>android.nonTransitiveRClass</code>	كل وحدة ترى موارد <code>R</code> الخاصة بها فقط، فيكون البناء أسرع

6. `app/src/main/AndroidManifest.xml`

الدور: بطاقة هوية التطبيق: الصلاحيات، والشاشات، والإعدادات العامة.

app/src/main/AndroidManifest.xml28 lines

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android">
3
4     <uses-permission android:name="android.permission.INTERNET" />
5
6     <!--
7         allowBackup="false": the encrypted token must not be copied to backups. The
8         encryption key stays in this device's Keystore, so a restored copy could not
9         be decrypted anyway.
10    -->
11     <application
12         android:name=".FakeStoreApp"
13         android:allowBackup="false"
```

```

14     android:label="@string/app_name"
15     android:supportsRtl="true"
16     android:theme="@style/Theme.FakeStore">
17
18     <activity
19         android:name=".MainActivity"
20         android:exported="true"
21         android:windowSoftInputMode="adjustResize">
22         <intent-filter>
23             <action android:name="android.intent.action.MAIN" />
24             <category android:name="android.intent.category.LAUNCHER" />
25         </intent-filter>
26     </activity>
27 </application>
28 </manifest>

```

الشرح

الصلاحيات والإعدادات:

- `INTERNET`: صلاحية الإنترنت. بدونها يفشل أي طلب شبكة.
- `android:name=".FakeStoreApp"`: كلاس الـ Application الذي يشغّل Hilt.
- `android:allowBackup="false"` (**قرار أمني**): يمنع نسخ ملفات التطبيق، ومنها الـ token المشفّر، إلى نسخ Google الاحتياطية. ومفتاح التشفير موجود في **Keystore** هذا الجهاز فقط، فالنسخة المستعادة على جهاز آخر لن تُفك أصلاً.
- `android:supportsRtl="true"`: دعم اللغات التي تُكتب من اليمين لليسار.

الشاشة:

- `MainActivity`: الشاشة الوحيدة (**Activity**).
- `exported="true"` مع `intent-filter` (MAIN + LAUNCHER): هذه الشاشة تظهر في قائمة التطبيقات وتفتح أولاً.
- `windowSoftInputMode="adjustResize"`: عند ظهور لوحة المفاتيح تقلص الشاشة، فلا تغطي حقول تسجيل الدخول.

الجزء الثاني: نماذج البيانات والـ APIs

7. data/model/Product.kt

الدور: شكل المنتج كما تراه الواجهة.

app/src/main/java/com/example/fakestore/data/model/Product.kt

15 lines

```
1 package com.example.fakestore.data.model
2
3 /**
4  * Product as the UI layer sees it: non-null, already cleaned up from the network DTO.
5  */
6 data class Product(
7     val id: Int,
8     val title: String,
9     val price: Double,
10    val description: String,
11    val category: String,
12    val imageUrl: String,
13    val rating: Double,
14    val ratingCount: Int,
15 )
```

الشرح

- `data class`: كلاس لحمل البيانات. يولّد Kotlin له تلقائياً `equals` و `hashCode` و `toString` و `copy`.
- `equals` مهمة لاحقاً: يستخدمها `DiffUtil` في الـ `RecyclerView` ليعرف هل تغيّر المنتج.
- **كل الحقول غير قابلة لأن تكون null**: الواجهة لا تتعامل أبداً مع قيم فارغة. التنظيف يحدث في الطبقة التي تحت الواجهة (القسم 10).

8. data/remote/ApiConfig.kt

الدور: عنوان السيرفر الأساسي في مكان واحد.

app/src/main/java/com/example/fakestore/data/remote/ApiConfig.kt

5 lines

```
1 package com.example.fakestore.data.remote
2
3 object ApiConfig {
4     const val BASE_URL = "https://fakestoreapi.com/"
5 }
```

الشرح

- `object`: كائن وحيد (**Singleton**) لا يُنشأ منه نسخ.
- `const val`: ثابت معروف وقت الترجمة.
- **مهم**: العنوان ينتهي بـ `/`، وهذا شرط عند Retrofit لدمجه مع مسارات مثل `products`.

الدور: شكل JSON المرسل والمستقبل في تسجيل الدخول.

app/src/main/java/com/example/fakestore/data/remote/dto/AuthDtos.kt

12 lines

```

1 package com.example.fakestore.data.remote.dto
2
3 /** Body of `POST /auth/login`. */
4 data class LoginRequest(
5     val username: String,
6     val password: String,
7 )
8
9 /** Response of `POST /auth/login`. Nullable because Gson can leave fields null. */
10 data class LoginResponse(
11     val token: String?,
12 )

```

الشرح

DTO اختصار **Data Transfer Object**: كلاس يطابق شكل JSON حرفياً.

الكلاس	يتحوّل إلى
LoginRequest	<code>{"username": "...", "password": "..."}"</code>
LoginResponse	<code>{"token": "..."}"</code> يُقرأ منه

لماذا `token: String?` قابل لأن يكون `null`؟ مكتبة **Gson** لا تحترم قواعد Kotlin. إذا غاب الحقل من JSON تضع فيه `null` حتى لو كان النوع غير قابل لذلك، فينهار التطبيق لاحقاً في مكان غير متوقع. لذلك نعلنه `String?` ونتحقق منه بأنفسنا في `AuthManager`.

الدور: شكل المنتج كما يأتي من السيرفر، ودالة تحويله إلى `Product`.

app/src/main/java/com/example/fakestore/data/remote/dto/ProductDto.kt

33 lines

```

1 package com.example.fakestore.data.remote.dto
2
3 import com.example.fakestore.data.model.Product
4
5 /**
6  * Product exactly as returned by the API. Every field is nullable because the server
7  * is outside our control; [toDomain] turns it into a safe [Product].
8  */
9 data class ProductDto(
10     val id: Int,
11     val title: String?,
12     val price: Double?,
13     val description: String?,
14     val category: String?,
15     val image: String?,
16     val rating: RatingDto?,

```

```

17 )
18
19 data class RatingDto(
20     val rate: Double?,
21     val count: Int?,
22 )
23
24 fun ProductDto.toDomain(): Product = Product(
25     id = id,
26     title = title.orEmpty(),
27     price = price ?: 0.0,
28     description = description.orEmpty(),
29     category = category.orEmpty(),
30     imageUrl = image.orEmpty(),
31     rating = rating?.rate ?: 0.0,
32     ratingCount = rating?.count ?: 0,
33 )

```

الشرح

- كل الحقول قابلة لأن تكون null، لأن السيرفر خارج سيطرتنا.
- دالة امتداد (**Extension Function**): تضيف دالة لكلاس موجود دون تعديله. تحوّل الـ DTO إلى `Product` نظيف:
 - `orEmpty()`: يحوّل `null` إلى نص فارغ.
 - `?:` (عامل **Elvis**): "إن كانت القيمة null فاستخدم البديل".
- اسم الحقل في JSON هو `image`، وعندنا `imageUrl`. هذا الفصل يحمي الواجهة من تغييرات السيرفر.

لماذا كلاسان (Model و DTO)؟ إذا غيّر السيرفر شكل الرد، نعدّل `ProductDto` و `toDomain()` فقط، والواجهة لا تتأثر.

11. data/remote/AuthApi.kt

الدور: تعريف طلب تسجيل الدخول.

app/src/main/java/com/example/fakestore/data/remote/AuthApi.kt

22 lines

```

1 package com.example.fakestore.data.remote
2
3 import com.example.fakestore.data.remote.dto.LoginRequest
4 import com.example.fakestore.data.remote.dto.LoginResponse
5 import retrofit2.Call
6 import retrofit2.http.Body
7 import retrofit2.http.POST
8
9 /**
10  * Authentication endpoint.
11  *
12  * Returns [Call] (not a suspend function) because the token refresh runs inside
13  * an OkHttp interceptor, which is synchronous and must block until the new token arrives.
14  *
15  * IMPORTANT: this API is built on an OkHttp client WITHOUT the AuthInterceptor,
16  * otherwise a refresh would trigger another refresh forever.
17  */
18 interface AuthApi {
19
20     @POST("auth/login")

```

```

21 fun login(@Body body: LoginRequest): Call<LoginResponse>
22 }

```

الشرح

- `@POST("auth/login")`: طلب POST إلى `https://fakestoreapi.com/auth/login`
- `@Body`: يحوّل `LoginRequest` إلى JSON ويضعه في جسم الطلب.

لماذا ترجع `Call<LoginResponse>` وليس دالة `suspend`؟ هذا قرار مقصود:

- التجديد يحدث داخل الـ `Interceptor`.
- الـ `Interceptor` دالة عادية متزامنة (**blocking**) تعمل على thread خاص بـ `OkHttp`، وليست `coroutine`.
- لذلك نحتاج طلباً ننفّذه وننتظر نتيجته مباشرة بـ `call.execute()`.

تحذير مكتوب في الكود: هذا الـ API يُبنى على عميل `OkHttp` بدون `AuthInterceptor`. لو مرّ طلب الـ `login` بالـ `Interceptor`، فسيطلب الـ `token` `Interceptor` صالحاً، فيبدأ تجديداً، والتجديد يستدعي `login` من جديد... حلقة لا نهائية.

12. data/remote/ProductApi.kt

الدور: طلبات المنتجات "المحمية".

app/src/main/java/com/example/fakestore/data/remote/ProductApi.kt

18 lines

```

1 package com.example.fakestore.data.remote
2
3 import com.example.fakestore.data.remote.dto.ProductDto
4 import retrofit2.http.GET
5 import retrofit2.http.Path
6
7 /**
8  * Protected product endpoints. Built on the authenticated OkHttp client, so every call
9  * automatically carries `Authorization: Bearer <token>`.
10 */
11 interface ProductApi {
12
13     @GET("products")
14     suspend fun getProducts(): List<ProductDto>
15
16     @GET("products/{id}")
17     suspend fun getProduct(@Path("id") id: Int): ProductDto
18 }

```

الشرح

- `@GET("products")`: يجلب قائمة المنتجات.
- `@GET("products/{id}")` مع `@Path("id")`: يضع رقم المنتج داخل المسار، مثل `products/5`.
- `suspend`: هنا مناسبة، لأن الـ `ViewModels` تستدعي هذه الدوال من `coroutines`.
- لا يوجد أي ذكر للـ `token` هنا. الـ `header` يُضاف تلقائياً بواسطة `AuthInterceptor`، لأن هذا الـ API مبني على العميل المحمي (القسم 26). هذا بالضبط معنى طلب المهمة: `"Attach Authorization: Bearer to all product requests"`.

الجزء الثالث ★: منطق الـ Token (قلب المهمة)

كل ملفات هذا الجزء في المجلد `data/auth/`، وتعمل معاً هكذا:

الملف	الدور
<code>Clock</code>	يعطي الوقت الحالي
<code>AuthSession</code>	ما نحفظه: الـ <code>token</code> ، و <code>tokenSavedAt</code> ، وبيانات الدخول
<code>TokenStorage</code>	واجهة التخزين، وتنفيذها <code>EncryptedTokenStorage</code> المشفّر
★ <code>AuthManager</code>	العقل: الدخول، والانتهاء، والتجديد، والقفل، والخروج
<code>AuthInterceptor</code>	يضيف <code>Bearer</code> لكل طلب، ويطلب <code>token</code> صالحاً من <code>AuthManager</code>
<code>TokenAuthenticator</code>	شبكة أمان عند رد 401
<code>AuthHeaders</code> / <code>AuthEvent</code> / <code>AuthExceptions</code>	أدوات مساعدة

13. `data/auth/Clock.kt`

الدور: مصدر الوقت الحالي.

app/src/main/java/com/example/fakestore/data/auth/Clock.kt

15 lines

```
1 package com.example.fakestore.data.auth
2
3 import javax.inject.Inject
4
5 /**
6  * Source of the current time. Injected so tests can move time forward instantly
7  * instead of waiting 60 real seconds for a token to expire.
8  */
9 fun interface Clock {
10     fun nowMillis(): Long
11 }
12
13 class SystemTimeClock @Inject constructor() : Clock {
14     override fun nowMillis(): Long = System.currentTimeMillis()
15 }
```

الشرح

- `fun interface`: واجهة فيها دالة واحدة فقط، تعيد الوقت بالميلي ثانية.
- `SystemTimeClock`: التنفيذ الحقيقي، يستخدم `System.currentTimeMillis()`.
- `@Inject constructor()`: يخبر Hilt أنه يستطيع إنشاء هذا الكلاس بنفسه.

لماذا لا نستدعي `System.currentTimeMillis()` مباشرة؟ حتى نختبر الكود. في الاختبارات نستبدل الساعة بـ `FakeClock` ونقدّم الوقت 61 ثانية فوراً، بدل الانتظار دقيقة كاملة في كل اختبار. هذا مثال عملي على فائدة حقن الاعتماديات (**Dependency Injection**).

لماذا `currentTimeMillis` وليس `SystemClock.elapsedRealtime` ؟ لأن `tokenSavedAt` يُحفظ على القرص ويبقى بعد إعادة تشغيل الجهاز، و `elapsedRealtime` يعود إلى الصفر عند إعادة التشغيل.

14. `data/auth/AuthSession.kt`

الدور: البيانات التي نحفظها بعد تسجيل الدخول.

app/src/main/java/com/example/fakestore/data/auth/AuthSession.kt

26 lines

```
1 package com.example.fakestore.data.auth
2
3 /**
4  * Username and password, kept only to simulate a token refresh (FakeStore has no
5  * refresh endpoint). A real backend would issue a refresh token instead.
6  */
7 data class Credentials(
8     val username: String,
9     val password: String,
10 ) {
11     override fun toString(): String = "Credentials(username=$username, password=*)"
12 }
13
14 /**
15  * Everything we persist after a successful login.
16  *
17  * @property savedAtMillis the `tokenSavedAt` timestamp used to compute expiry.
18  */
19 data class AuthSession(
20     val token: String,
21     val savedAtMillis: Long,
22     val credentials: Credentials,
23 ) {
24     override fun toString(): String =
25         "AuthSession(token=*, savedAtMillis=$savedAtMillis, credentials=$credentials)"
26 }
```

الشرح

`Credentials` : اسم المستخدم وكلمة المرور.

- نحتاجهما لأن **التجديد = تسجيل دخول جديد**، ف FakeStore لا يملك endpoint للتجديد.

`: AuthSession`

- `token` : الرمز نفسه.
- `savedAtMillis` : هو `tokenSavedAt` المطلوب في المهمة.
- `credentials` : بيانات الدخول.

تفصيل أمني مهم: `override fun toString()`

- `data class` يطبع كل حقوله تلقائياً عند الطباعة.
- لو كتب أحد `Log.d("x", session.toString())` لظهرت كلمة المرور وال token في Logcat.
- أعدنا تعريف `toString` لطبع `***` بدلاً منهما.

الدور: واجهة (interface) للتخزين: اقرأ، احفظ، امسح.

app/src/main/java/com/example/fakestore/data/auth/TokenStorage.kt

11 lines

```

1 package com.example.fakestore.data.auth
2
3 /**
4  * Persistence for the current [AuthSession]. Implementations must be thread-safe:
5  * several OkHttp threads may read while a refresh writes.
6  */
7 interface TokenStorage {
8     fun read(): AuthSession?
9     fun save(session: AuthSession)
10    fun clear()
11 }

```

الشرح

لماذا interface؟ يوجد تنفيذان:

التنفيذ	أين يُستخدم
EncryptedTokenStorage	في التطبيق: مشقّر وعلى القرص
InMemoryTokenStorage	في الاختبارات: في الذاكرة فقط

و `AuthManager` لا يعرف ولا يهتم أيهما يستخدم.

شرط أمان الخيوط (Thread-safety): التطبيق يجب أن يكون آمناً عند وصول عدة threads في نفس اللحظة. OkHttp يقرأ من عدة threads، والتجديد يكتب في نفس الوقت.

الدور: تحقيق متطلب "Securely store the token".

app/src/main/java/com/example/fakestore/data/auth/EncryptedTokenStorage.kt

63 lines

```

1 package com.example.fakestore.data.auth
2
3 import android.content.Context
4 import android.content.SharedPreferences
5 import androidx.security.crypto.EncryptedSharedPreferences
6 import androidx.security.crypto.MasterKey
7 import dagger.hilt.android.qualifiers.ApplicationContext
8 import javax.inject.Inject
9 import javax.inject.Singleton
10
11 /**
12  * [TokenStorage] backed by EncryptedSharedPreferences: keys and values are encrypted
13  * with AES-256, and the master key lives in the Android Keystore, so it never leaves the device.
14  */
15 @Singleton
16 class EncryptedTokenStorage @Inject constructor(
17     @ApplicationContext private val context: Context,

```

```

18 ) : TokenStorage {
19
20     private val prefs: SharedPreferences by lazy {
21         val masterKey = MasterKey.Builder(context)
22             .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
23             .build()
24
25         EncryptedSharedPreferences.create(
26             context,
27             FILE_NAME,
28             masterKey,
29             EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
30             EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM,
31         )
32     }
33
34     override fun read(): AuthSession? {
35         val token = prefs.getString(KEY_TOKEN, null) ?: return null
36         val username = prefs.getString(KEY_USERNAME, null) ?: return null
37         val password = prefs.getString(KEY_PASSWORD, null) ?: return null
38         val savedAt = prefs.getLong(KEY_SAVED_AT, 0L)
39         return AuthSession(token, savedAt, Credentials(username, password))
40     }
41
42     override fun save(session: AuthSession) {
43         // commit() writes synchronously; we are always on a background thread here.
44         prefs.edit()
45             .putString(KEY_TOKEN, session.token)
46             .putLong(KEY_SAVED_AT, session.savedAtMillis)
47             .putString(KEY_USERNAME, session.credentials.username)
48             .putString(KEY_PASSWORD, session.credentials.password)
49             .commit()
50     }
51
52     override fun clear() {
53         prefs.edit().clear().commit()
54     }
55
56     private companion object {
57         const val FILE_NAME = "secure_auth_prefs"
58         const val KEY_TOKEN = "token"
59         const val KEY_SAVED_AT = "token_saved_at"
60         const val KEY_USERNAME = "username"
61         const val KEY_PASSWORD = "password"
62     }
63 }

```

الشرح

MasterKey: المفتاح الرئيسي للتشفير.

- AES256_GCM**: خوارزمية تشفير قوية.

- المفتاح يُنشأ ويُحفظ داخل **Android Keystore**. وهي خزانة في نظام Android لا يمكن إخراج المفاتيح منها، ولا حتى للتطبيق نفسه. التطبيق يطلب منها "شفر" و"فك" فقط.

EncryptedSharedPreferences.create(...): نسخة مشفرة من **SharedPreferences**:

- AES256_SIV**: يشفر **أسماء** المفاتيح، فلا يُعرف حتى أن هناك حقلاً اسمه "token".

- AES256_GCM**: يشفر **القيم**.

by lazy: يُنشأ الكائن عند أول استخدام فقط، وليس عند إنشاء الكلاس، لأن إنشاءه يتعامل مع الـ Keystore وهو مكلف نسبياً.

`: read()`

- يقرأ الحقول الأربعة.
- إذا غاب أي منها يُرجع `null`، أي لا توجد جلسة.
- `?: return null` : إن كانت القيمة `null` فاخرج من الدالة فوراً.

`: save()`

- يكتب الحقول الأربعة دفعة واحدة.
- يستخدم `commit()` : يكتب على القرص **وينتظر** حتى ينتهي.
- البديل `apply()` يكتب في الخلفية. اخترنا `commit` لأننا دائماً على `thread` خلفي، فلا مشكلة في الانتظار، ونريد ضمان الحفظ.

`clear()` : يسمح كل شيء، ويُستخدم عند تسجيل الخروج.

ملاحظة للمقابلة: Google أوقفت تطوير مكتبة `security-crypto` (أصبحت **deprecated**). البديل الحديث هو **DataStore** مع تشفير بمفتاح من `Keystore` (مثلاً بمكتبة **Tink**). وبفضل `TokenStorage` ك `interface`، يكفي كتابة كلاس جديد وتغيير سطر واحد في `DataModule`، دون لمس أي كلاس آخر.

17. `data/auth/AuthExceptions.kt`

الدور: أنواع الأخطاء الخاصة بالمصادقة.

app/src/main/java/com/example/fakestore/data/auth/AuthExceptions.kt

17 lines

```
1 package com.example.fakestore.data.auth
2
3 import java.io.IOException
4
5 /*
6  * Both exceptions extend IOException on purpose: they can be thrown from inside an
7  * OkHttp interceptor, and OkHttp only reports IOExceptions as normal call failures.
8  * Any other exception type thrown there would crash the app.
9  */
10
11 /** The user is not logged in anymore (no session, or the refresh was rejected). */
12 class SessionExpiredException(cause: Throwable? = null) :
13     IOException("Session expired. Please log in again.", cause)
14
15 /** The server rejected the username/password (HTTP 4xx from /auth/login). */
16 class InvalidCredentialsException(val httpCode: Int) :
17     IOException("Invalid username or password (HTTP $httpCode).")
```

الشرح

SessionExpiredException : لا توجد جلسة، أو السيرفر رفض التجديد. النتيجة: يجب تسجيل الدخول من جديد.

InvalidCredentialsException : السيرفر رفض اسم المستخدم أو كلمة المرور، أي رد بكود **4xx** مثل 401. نحتفظ بالكود في `httpCode`.

⚠ **لماذا يرثان من `IOException` ؟** هذه نقطة تقنية دقيقة ومهمة:

- هذه الأخطاء قد تُرمى من داخل الـ **Interceptor**.

- OkHttp يتعامل مع `IOException` كفشل طبيعي للطلب، فيمرره لـ Retrofit ثم لـ `ViewModel`، فيظهر للمستخدم كرسالة خطأ.
- أي نوع آخر (مثل `IllegalStateException`) يُعتبر خطأً برمجياً، فينهار التطبيق كاملاً (**crash**) لأنه يُرمى على thread خاص بـ OkHttp.

18. data/auth/AuthEvent.kt

الدور: أحداث لمرة واحدة يرسلها `AuthManager` للواجهة.

```
app/src/main/java/com/example/fakestore/data/auth/AuthEvent.kt 7 lines

1 package com.example.fakestore.data.auth
2
3 /** One-time events emitted by [AuthManager] and observed by the UI. */
4 sealed interface AuthEvent {
5     /** Refresh failed and the session was wiped: the UI must return to the login screen. */
6     data object SessionExpired : AuthEvent
7 }
```

الشرح

- `sealed interface`: مجموعة مغلقة من الأنواع. حالياً نوع واحد هو `SessionExpired`، ويمكن إضافة أنواع لاحقاً.
- `data object`: كائن وحيد لا يحمل بيانات.

لماذا "حدث" (Event) وليس "حالة" (State)؟

- تسجيل الخروج الإجباري شيء يحدث مرة واحدة: نعرض رسالة ثم ننتقل لشاشة Login.
- لو كان "حالة" لبقى محفوظاً، ولتكرر الانتقال عند كل تدوير للشاشة.

19. data/auth/AuthHeaders.kt

الدور: ثوابت ودوال لبناء header الـ Authorization وقراءته.

```
app/src/main/java/com/example/fakestore/data/auth/AuthHeaders.kt 12 lines

1 package com.example.fakestore.data.auth
2
3 object AuthHeaders {
4     const val AUTHORIZATION = "Authorization"
5     private const val BEARER_PREFIX = "Bearer "
6
7     fun bearer(token: String): String = BEARER_PREFIX + token
8
9     /** Extracts the token from `Bearer <token>`, or null if the header is missing. */
10    fun tokenFrom(headerValue: String?): String? =
11        headerValue?.takeIf { it.startsWith(BEARER_PREFIX) }?.removePrefix(BEARER_PREFIX)
12 }
```

الشرح

- `bearer(token)`: يبني النص `Bearer abc123`.

- `tokenFrom(header)`: العكس، يستخرج `abc123` من `Bearer abc123`. يستخدمه `TokenAuthenticator` ليعرف أي `token` رفضه السيرفر.
 - `takeIf { ... }`: يُرجع القيمة إذا تحقق الشرط، وإلا يُرجع `null`.
- لماذا ملف خاص لهذه الثوابت؟** حتى لا يُكتب النص `"Authorization"` و `"Bearer "` في عدة أماكن. خطأ إملائي واحد، مثل نسيان المسافة بعد `Bearer`، يكسر المصادقة كلها.

20. ★★ `data/auth/AuthManager.kt`

الدور: أهم ملف في المهمة. المسؤول الوحيد عن حالة المصادقة.

app/src/main/java/com/example/fakestore/data/auth/AuthManager.kt

163 lines

```
1 package com.example.fakestore.data.auth
2
3 import com.example.fakestore.data.remote.AuthApi
4 import com.example.fakestore.data.remote.dto.LoginRequest
5 import kotlinx.coroutines.Dispatchers
6 import kotlinx.coroutines.flow.MutableSharedFlow
7 import kotlinx.coroutines.flow.SharedFlow
8 import kotlinx.coroutines.flow.asSharedFlow
9 import kotlinx.coroutines.withContext
10 import java.io.IOException
11 import javax.inject.Inject
12 import javax.inject.Singleton
13
14 /**
15  * Single owner of the authentication state: login, token expiry, refresh and logout.
16  *
17  * Token rules (simulated, because FakeStore tokens never really expire):
18  * - A token is valid for [TOKEN_LIFETIME_MS] after it was saved (`tokenSavedAt`).
19  * - An expired token is never handed out: it is refreshed first.
20  * - Refresh = calling `/auth/login` again with the stored credentials.
21  * - Concurrent callers share ONE refresh: the first one refreshes while the others
22  *   wait on [refreshLock], then reuse the new token (double-checked locking).
23  * - If the server rejects the refresh, the user is logged out ([AuthEvent.SessionExpired]).
24  */
25 @Singleton
26 class AuthManager @Inject constructor(
27     private val authApi: AuthApi,
28     private val storage: TokenStorage,
29     private val clock: Clock,
30 ) {
31
32     /** Serializes refreshes: only one thread at a time may request a new token. */
33     private val refreshLock = Any()
34
35     /**
36      * Guards writes to [storage] together with [sessionGeneration]. Held only briefly
37      * (never during network calls), so logout() on the main thread never waits for a refresh.
38      */
39     private val storageLock = Any()
40
41     /** Incremented on every logout, so a refresh that started before it cannot revive the session. */
42     private var sessionGeneration = 0
43
44     private val _events = MutableSharedFlow<AuthEvent>(extraBufferCapacity = 1)
45     val events: SharedFlow<AuthEvent> = _events.asSharedFlow()
46
47     fun isLoggedIn(): Boolean = storage.read() != null
```

```

48
49  /**
50   * Logs in from the UI and stores the new session.
51   *
52   * @throws InvalidCredentialsException if the server rejects the username/password.
53   * @throws IOException on network or server errors.
54   */
55  suspend fun login(username: String, password: String) {
56      withContext(Dispatchers.IO) {
57          val generation = currentGeneration()
58          val session = requestNewSession(Credentials(username, password))
59          saveIfStillCurrent(generation, session)
60      }
61  }
62
63  /**
64   * Returns a token that is safe to send right now, refreshing it first if it expired.
65   *
66   * Blocking: called by [AuthInterceptor] on an OkHttp background thread.
67   *
68   * @throws SessionExpiredException if there is no session or the refresh was rejected.
69   * @throws IOException if the refresh failed because of the network (session is kept).
70   */
71  fun getValidToken(): String {
72      // Fast path: no locking when the token is still valid (the common case).
73      storage.read()?.takeUnless { it.isExpired() }?.let { return it.token }
74
75      synchronized(refreshLock) {
76          // Second check: while we waited for the lock, another thread may have refreshed.
77          val session = storage.read() ?: throw SessionExpiredException()
78          if (!session.isExpired()) return session.token
79          return refresh(session.credentials)
80      }
81  }
82
83  /**
84   * Called by [TokenAuthenticator] when the server answered 401 to [rejectedToken].
85   *
86   * @return a token to retry with, or null to give up.
87   */
88  fun refreshAfterUnauthorized(rejectedToken: String?): String? {
89      synchronized(refreshLock) {
90          val session = storage.read() ?: return null
91          // Another request already replaced the rejected token: just retry with the new one.
92          if (session.token != rejectedToken) return session.token
93          return try {
94              refresh(session.credentials)
95          } catch (e: IOException) {
96              null
97          }
98      }
99  }
100
101  /** User-initiated logout. */
102  fun logout() {
103      synchronized(storageLock) {
104          sessionGeneration++
105          storage.clear()
106      }
107  }
108
109  /** Logout caused by a failed refresh: clears the session and tells the UI. */
110  fun forceLogout() {
111      logout()
112      _events.tryEmit(AuthEvent.SessionExpired)
113  }

```

```

114
115 /** Must be called while holding [refreshLock]. */
116 private fun refresh(credentials: Credentials): String {
117     val generation = currentGeneration()
118     val newSession = try {
119         requestNewSession(credentials)
120     } catch (e: InvalidCredentialsException) {
121         // The server refused our credentials: retrying would fail forever, so log out.
122         forceLogout()
123         throw SessionExpiredException(e)
124     }
125     saveIfStillCurrent(generation, newSession)
126     return newSession.token
127 }
128
129 /** Blocking network call to `/auth/login`. */
130 private fun requestNewSession(credentials: Credentials): AuthSession {
131     val response = authApi
132         .login(LoginRequest(credentials.username, credentials.password))
133         .execute()
134
135     if (response.code() in 400..499) throw InvalidCredentialsException(response.code())
136     if (!response.isSuccessful) throw IOException("Login failed with HTTP ${response.code()}")
137
138     val token = response.body()?.token
139     if (token.isNullOrEmpty()) throw IOException("Login response did not contain a token")
140
141     return AuthSession(token, clock.nowMillis(), credentials)
142 }
143
144 private fun currentGeneration(): Int = synchronized(storageLock) { sessionGeneration }
145
146 /** Saves [session] unless a logout happened since [generation] was read. */
147 private fun saveIfStillCurrent(generation: Int, session: AuthSession) {
148     synchronized(storageLock) {
149         if (generation != sessionGeneration) throw SessionExpiredException()
150         storage.save(session)
151     }
152 }
153
154 private fun AuthSession.isExpired(): Boolean {
155     val age = clock.nowMillis() - savedAtMillis
156     // A negative age means the device clock moved backwards: don't trust the token.
157     return age < 0 || age >= TOKEN_LIFETIME_MS
158 }
159
160 companion object {
161     const val TOKEN_LIFETIME_MS = 60_000L
162 }
163 }

```

الشرح التفصيلي

أ) الإعلان والاعتماديات (الأسطر 25-30)

- `@Singleton`: نسخة واحدة فقط في التطبيق كله. ضروري، لأن القفل يجب أن يكون واحداً لكل الطلبات. لو وُجدت نسختان لكان لكل منهما قفلها، ولفشل منع التجديد المكرر.
- يحصل على 3 أشياء من الخارج (حقن الاعتماديات):
 - `authApi`: لإرسال `.login`.
 - `storage`: للحفظ.

○ clock : للوقت.

ب) الأقفال والحالة (الأسطر 33-45)

refreshLock : القفل الأساسي.

- يضمن أن thread واحد فقط يجدد token في المرة الواحدة.
- Any() كائن فارغ نستخدمه كمفتاح للقفل.
- storageLock و sessionGeneration : لحل مشكلة دقيقة:
- السيناريو: بدأ تجديد، والمستخدم ضغط Logout أثناء انتظار رد السيرفر، ثم وصل الرد بـ token جديد.
- بدون حماية: يُحفظ token الجديد بعد الخروج، فيعود المستخدم مسجلاً للدخول رغم أنه خرج!
- الحل: رقم "جيل" (sessionGeneration) يزيد بواحد عند كل Logout.
- 1. قبل التجديد نحفظ رقم الجيل الحالي.
- 2. بعد الرد نقارنه بالرقم الحالي.
- 3. إذا تغير فهذا يعني أن خروجاً حدث، فلا نحفظ.
- storageLock يُمسك لجزء صغير جداً من الثانية فقط (بلا أي طلب شبكة)، فلا تنتظر الواجهة عند Logout.
- _events و events :
- MutableSharedFlow : قناة بث للأحداث. خاصة وقابلة للإرسال.
- events : النسخة العامة، للقراءة فقط. MainActivity تستمع لها.
- extraBufferCapacity = 1 : مكان لحدث واحد ينتظر، فلا يضيع إذا أرسل والمستمع مشغول.

ج) isLoggedIn() (السطر 47)

هل توجد جلسة محفوظة؟ نستخدمها MainActivity لتقرر أي شاشة تبدأ بها.

د) login() : تسجيل الدخول من الشاشة (الأسطر 55-61)

- دالة suspend ، تستدعيها LoginViewModel من coroutine.
- withContext(Dispatchers.IO) : ينقل التنفيذ إلى thread مخصص لعمليات الشبكة والقرص، لأن requestNewSession تنتظر الرد (blocking). لو نُفذت على الـ Main thread لتجمّد التطبيق، ورمي Android الخطأ NetworkOnMainThreadException.
- الخطوات: نأخذ رقم الجيل، ثم نطلب جلسة جديدة، ثم نحفظها إن لم يحدث خروج في الأثناء.

هـ) ★ getValidToken() : الدالة الأهم (الأسطر 71-81)

تُستدعى من AuthInterceptor قبل كل طلب منتجات. تمر بثلاث مراحل:

المرحلة 1: المسار السريع (السطر 73)، بلا قفل

```
storage.read()?.takeUnless { it.isExpired() }?.let { return it.token }
```

اقرأ الجلسة. إذا لم تكن منتهية فأرجع الـ token فوراً.

- { شرط } takeUnless : يُرجع القيمة إلا إذا تحقق الشرط.
- هذه الحالة الأكثر شيوعاً (الـ token صالح)، فلا نريد أن تنتظر الطلبات في طابور القفل بلا داعٍ.

المرحلة 2: دخول القفل (السطر 75) `synchronized(refreshLock) { ... }` : هنا يقف الطابور. أول thread يدخل، والبقية ينتظرون خارج القفل حتى يخرج.

المرحلة 3: التحقق الثاني داخل القفل (الأسطر 77-79)، سر الحل

- السطر 77: نقرأ الجلسة **من جديد**. إذا لم توجد (خرج المستخدم) نرمي `SessionExpiredException`.
- السطر 78: إذا أصبح الـ **token صالحاً الآن** نرجعه دون تجديد. هذا يعني أن thread آخر دخل القفل قبلنا وجده بينما كنا ننتظر.
- السطر 79: ما زال منتهياً، فنحن أول من وصل، **فنجدد**.

مثال بثلاثة طلبات A و B و C والـ token منته:

اللحظة	A	B	C
1	المسار السريع: منته	المسار السريع: منته	المسار السريع: منته
2	يدخل القفل 🔒	ينتظر ⌚	ينتظر ⌚
3	التحقق الثاني: منته، فيجدد (login)	ينتظر ⌚	ينتظر ⌚
4	يحفظ الجديد ويخرج 🔒	يدخل، والتحقق الثاني: صالح!	ينتظر ⌚
5	يرسل طلبه ✅	يخرج ويرسل ✅	يدخل، صالح! فيرسل ✅

النتيجة: **login واحد فقط**. أثبتنا ذلك في الاختبارات (القسم 51): بدون السطر 78، أنتجت 10 طلبات متزامنة 10 طلبات login. لماذا `synchronized` وليس `Mutex` من **coroutines**؟ الـ `Interceptor` ليس `coroutine`، بل دالة عادية تعمل على thread حقيقي من `OkHttp`، و `synchronized` هو القفل المناسب للـ `threads` الحقيقية. `Mutex` يحتاج دالة `suspend`، فكنا سنضطر إلى `runBlocking`، وهو أقل وضوحاً.

و) (`refreshAfterUnauthorized()` (الأسطر 88-99)

يستخدمها `TokenAuthenticator` عندما يرد السيرفر 401:

- السطر 90: لا توجد جلسة، فنرجع `null`، أي استسلم.
- السطر 92: إذا كان الـ `token` المحفوظ **يختلف** عن الذي رُفض، فطلب آخر جده قبلنا. نرجع الجديد دون تجديد إضافي.
- غير ذلك: نجدد. وإذا فشل التجديد نرجع `null`.

ز) (`logout()` و `forceLogout()` (الأسطر 102-113)

`logout()`: خروج عادي بطلب المستخدم.

- يزيد رقم الجيل بواحد، ثم يمسح التخزين.
- كلاهما داخل `storageLock` ليحدثا معاً خطوة واحدة.

`forceLogout()`: خروج إجباري.

- يفعل نفس الشيء، ثم **يرسل حدث** `SessionExpired` للواجهة.
- `tryEmit`: يرسل دون انتظار، لأننا قد نكون على thread خاص بـ `OkHttp`.

ح) (`refresh()` (الأسطر 116-127)

- تُستدعى **فقط** وهي ممسكة بـ `refreshLock`.

- تطلب جلسة جديدة بنفس بيانات الدخول المحفوظة.
- إذا رفض السيرفر بيانات الدخول (`InvalidCredentialsException`):
 1. تسجيل خروج إجباري (`forceLogout`).
 2. نرمي `SessionExpiredException`.
 3. لا نعيد المحاولة، فالمحاولة الثانية ستفشل أيضاً، وإعادتها تعني حلقة لا نهائية. هذا تنفيذ لتحذير المهمة: "Avoid infinite retry loops" و "If refresh fails → force logout".
- إذا كان الفشل بسبب الشبكة (`IOException`): الخطأ يمر للطلب كما هو، والجلسة تبقى. انقطاع الإنترنت لحظياً لا يجب أن يُخرج المستخدم، والمحاولة التالية ستجدد عندما يعود الإنترنت.

(ط) `requestNewSession()` (الأسطر 130-142)

الطلب الفعلي إلى `/auth/login`:

- `execute()`: ينفذ الطلب **وينتظر** الرد (blocking).
- **السطر 135**: كود من 400 إلى 499، أي أن السيرفر رفض البيانات، فنرمي `InvalidCredentialsException`.
- **السطر 136**: أي فشل آخر (مثل 500، أي عطل في السيرفر) نرمي `IOException` عادياً. ليس ذنب المستخدم، فلا نُخرجه.
- **السطر 139**: نجاح لكن بلا token (رد غريب) نرمي `IOException` أيضاً.
- **السطر 141**: ننشئ الجلسة مع `clock.nowMillis()`، وهذا هو `tokenSavedAt`.

(ي) `isExpired()` (الأسطر 154-158)

```
age = now - savedAtMillis
expired = age < 0 || age >= 60_000
```

- `age >= 60_000`: مرت 60 ثانية أو أكثر، فال token **منتهٍ**. هذا متطلب "Treat token as expired after 60 seconds".
- `age < 0`: الوقت الحالي **قبل** وقت الحفظ. يحدث هذا إذا غيّر المستخدم ساعة الجهاز للخلف. لا ننق بال token، فنعتبره منتهياً ونجّده. حالة حدّية (edge case) نادرة، لكن معالجتها تُظهر الاحترافية.

(ك) `TOKEN_LIFETIME_MS` (السطر 161)

مدة الصلاحية = `60_000L` ميلي ثانية = **60 ثانية**. الشرطة السفلية `_` لتسهيل قراءة الأرقام فقط.

21. `data/auth/AuthInterceptor.kt`

الدور: متطلب "Authorization Interceptor".

app/src/main/java/com/example/fakestore/data/auth/AuthInterceptor.kt

27 lines

```
1 package com.example.fakestore.data.auth
2
3 import okhttp3.Interceptor
4 import okhttp3.Response
5 import javax.inject.Inject
6
7 /**
8  * Adds `Authorization: Bearer <token>` to every request of the authenticated client.
9  *
10 * [AuthManager.getValidToken] guarantees the token is not expired: if it is, the call
11 * blocks here until the (single, shared) refresh finishes. A request is therefore never
```

```

12  * sent with an expired token.
13  */
14  class AuthInterceptor @Inject constructor(
15      private val authManager: AuthManager,
16  ) : Interceptor {
17
18      override fun intercept(chain: Interceptor.Chain): Response {
19          val token = authManager.getValidToken()
20
21          val authorizedRequest = chain.request().newBuilder()
22              .header(AuthHeaders.AUTHORIZATION, AuthHeaders.bearer(token))
23              .build()
24
25          return chain.proceed(authorizedRequest)
26      }
27  }

```

الشرح

يرث `Interceptor` من `OkHttp`، وفيه دالة واحدة هي `intercept(chain)`، يستدعيها `OkHttp` لكل طلب يمر بالعميل المحمي:

1. `authManager.getValidToken()`: اطلب `token` صالحاً. - إذا كان صالحاً يعود فوراً. - إذا كان منتهياً **ينتظر هنا** حتى ينتهي التجديد (واحد مشترك بين كل الطلبات). - النتيجة: **لا يُرسل أي طلب بـ `token` منتهٍ أبداً**. هذا متطلب *"Do not send requests with an expired token"*.
2. `chain.request().newBuilder()`: الطلبات في `OkHttp` غير قابلة للتعديل (`immutable`)، فننشئ نسخة جديدة منها.
3. `header("Authorization", "Bearer ...")`: نضيف الـ `header`. تستبدل أي قيمة سابقة بنفس الاسم، بخلاف `addHeader` التي تضيف نسخة ثانية.
4. `chain.proceed(request)`: نمرر الطلب للحلقة التالية في السلسلة (`Interceptor` آخر أو الشبكة)، ونرجع الرد.

تشبيه: موظف عند باب الشركة يختم كل رسالة خارجة. إذا كان الختم منتهياً يطلب ختماً جديداً أولاً، وإذا جاء معه زملاء في نفس اللحظة ينتظرون الختم الجديد ولا يطلب كل منهم ختماً.

22. `data/auth/TokenAuthenticator.kt`

الدور: شبكة أمان إضافية عند رد `401 Unauthorized`.

app/src/main/java/com/example/fakestore/data/auth/TokenAuthenticator.kt

35 lines

```

1  package com.example.fakestore.data.auth
2
3  import okhttp3.Authenticator
4  import okhttp3.Request
5  import okhttp3.Response
6  import okhttp3.Route
7  import javax.inject.Inject
8
9  /**
10   * Safety net for a real backend: OkHttp calls this when the server answers 401.
11   * (FakeStore never does, because it does not check tokens; expiry is handled up front
12   * by [AuthInterceptor].)
13   *
14   * Retries at most ONCE per request, so a server that keeps rejecting us can never
15   * cause an infinite refresh loop.

```

```

16  */
17  class TokenAuthenticator @Inject constructor(
18      private val authManager: AuthManager,
19  ) : Authenticator {
20
21      override fun authenticate(route: Route?, response: Response): Request? {
22          // priorResponse != null means this request was already retried once and failed again.
23          if (response.priorResponse != null) {
24              authManager.forceLogout()
25              return null
26          }
27
28          val rejectedToken = AuthHeaders.tokenFrom(response.request.header(AuthHeaders.AUTHORIZATION))
29          val newToken = authManager.refreshAfterUnauthorized(rejectedToken) ?: return null
30
31          return response.request.newBuilder()
32              .header(AuthHeaders.AUTHORIZATION, AuthHeaders.bearer(newToken))
33              .build()
34      }
35  }

```

الشرح

ما هو Authenticator في OkHttp؟ كلاس يستدعيه OkHttp تلقائياً عندما يرد السيرفر بالكود 401، أي "غير مصادَق".

- إذا أُرْجِع طلباً جديداً، يعيد OkHttp المحاولة به.
- إذا أُرْجِع `null`، يتوقف ويُرجع رد الـ 401 كما هو.

لماذا نحتاجه إذا كان FakeStore لا يرد بـ 401 أبداً؟ في السيرفرات الحقيقية قد يرفض السيرفر token نعتبره صالحاً، مثلاً إذا ألغاه المسؤول أو اختلفت الساعات. هذا الكلاس يُظهر للمراجع أنك تعرف كيف تُبنى المصادقة في التطبيقات الحقيقية.

متى يعمل	الأسلوب	
قبل الإرسال	استباقي (Proactive)	AuthInterceptor
بعد رد 401	تفاعلي (Reactive)	TokenAuthenticator

المنطق خطوة بخطوة:

1. `response.priorResponse != null`: هذا الطلب أُعيدت محاولته مرة من قبل ورفض مرة ثانية. نسجّل خروجاً إجبارياً ونرجع `null`. هكذا لا تتكرر المحاولة أكثر من مرة، فلا توجد حلقة لا نهائية.
2. نستخرج الـ token المرفوض من الطلب.
3. نطلب من `AuthManager` تجديداً، وهو يتحقق أولاً هل جدّد طلب آخر قبلنا.
4. نعيد بناء الطلب بالـ token الجديد، فيرسله OkHttp مرة ثانية.

الجزء الرابع: ال Repository وحقن الاعتماديات (Hilt)

Dependency Injection (حقن الاعتماديات): بدل أن ينشئ كل كلاس ما يحتاجه بنفسه، يُعطى له من الخارج.

تشبيه: الطاهي لا يزرع الخضار بنفسه، بل يستلمها جاهزة من المورد. **Hilt** هو "المورد": يعرف كيف يصنع كل شيء، ويوصله لمن يحتاجه.

23. FakeStoreApp.kt

الدور: كلاس التطبيق، وأول ما يُنشأ عند التشغيل.

app/src/main/java/com/example/fakestore/FakeStoreApp.kt

10 lines

```
1 package com.example.fakestore
2
3 import android.app.Application
4 import dagger.hilt.android.HiltAndroidApp
5
6 /**
7  * Application entry point. [HiltAndroidApp] generates the app-wide dependency container.
8  */
9 @HiltAndroidApp
10 class FakeStoreApp : Application()
```

الشرح

- `@HiltAndroidApp`: يجعل Hilt يولّد "الحاوية" الرئيسية للاعتماديات، التي تعيش طوال عمر التطبيق.
- بدونها لا يعمل أي `@Inject` أو `@AndroidEntryPoint` في المشروع.
- مسجّل في ال Manifest عبر `android:name`.

24. data/repository/ProductRepository.kt

الدور: المصدر الوحيد لبيانات المنتجات بالنسبة لل ViewModels.

app/src/main/java/com/example/fakestore/data/repository/ProductRepository.kt

21 lines

```
1 package com.example.fakestore.data.repository
2
3 import com.example.fakestore.data.model.Product
4 import com.example.fakestore.data.remote.ProductApi
5 import com.example.fakestore.data.remote.dto.toDomain
6 import javax.inject.Inject
7
8 /** Single source of product data for the ViewModels. */
9 interface ProductRepository {
10     suspend fun getProducts(): List<Product>
11     suspend fun getProduct(id: Int): Product
12 }
13
14 class ProductRepositoryImpl @Inject constructor(
15     private val api: ProductApi,
16 ) : ProductRepository {
```

```

17
18     override suspend fun getProducts(): List<Product> = api.getProducts().map { it.toDomain() }
19
20     override suspend fun getProduct(id: Int): Product = api.getProduct(id).toDomain()
21 }

```

الشرح

- **interface** للفصل: ال ViewModel يعرف `ProductRepository` فقط، ولا يعرف Retrofit.
 - `ProductRepositoryImpl`: يستدعي ال API ويحوّل كل DTO إلى `Product` عبر `toDomain()`.
 - `map { it.toDomain() }`: يحوّل كل عنصر في القائمة.
 - `@Inject constructor(private val api: ProductApi)`: Hilt يعطيه `ProductApi` جاهزاً، ومعه العميل المحمي وال `Interceptor`.
- هذه طبقة ال Model في MVVM. لو أضفنا لاحقاً تخزيناً محلياً (Room) للعمل بلا إنترنت، نعدّل هذا الكلاس فقط.

25. di/Qualifiers.kt

الدور: تسميتان لتمييز عمليتي OkHttpClient.

app/src/main/java/com/example/fakestore/di/Qualifiers.kt

13 lines

```

1 package com.example.fakestore.di
2
3 import javax.inject.Qualifier
4
5 /** OkHttpClient WITHOUT auth: used only for `/auth/login`. */
6 @Qualifier
7 @Retention(AnnotationRetention.BINARY)
8 annotation class PlainClient
9
10 /** OkHttpClient WITH AuthInterceptor + TokenAuthenticator: used for protected APIs. */
11 @Qualifier
12 @Retention(AnnotationRetention.BINARY)
13 annotation class AuthClient

```

الشرح

المشكلة: عندنا كائنان من نفس النوع `OkHttpClient`، وHilt يميّز الأشياء بنوعها، فلن يعرف أيهما يعطي.

الحل: `@Qualifier`، أي annotation نصنعها لتكون "ملصقاً" يميّز بينهما:

العمليل	الملصق
بلا مصادقة، لطلب login فقط	<code>@PlainClient</code>
مع <code>AuthInterceptor</code> و <code>TokenAuthenticator</code> ، للمنتجات	<code>@AuthClient</code>

`@Retention(BINARY)`: الملصق محفوظ في الكود المترجم، وهذا ما يحتاجه Hilt.

الدور: "وصفة" بناء كل ما يخص الشبكة، وهنا يتحقق فصل العميلين.

app/src/main/java/com/example/fakestore/di/NetworkModule.kt

84 lines

```

1 package com.example.fakestore.di
2
3 import com.example.fakestore.BuildConfig
4 import com.example.fakestore.data.auth.AuthHeaders
5 import com.example.fakestore.data.auth.AuthInterceptor
6 import com.example.fakestore.data.auth.TokenAuthenticator
7 import com.example.fakestore.data.remote.ApiConfig
8 import com.example.fakestore.data.remote.AuthApi
9 import com.example.fakestore.data.remote.ProductApi
10 import dagger.Module
11 import dagger.Provides
12 import dagger.hilt.InstallIn
13 import dagger.hilt.components.SingletonComponent
14 import okhttp3.OkHttpClient
15 import okhttp3.logging.HttpLoggingInterceptor
16 import retrofit2.Retrofit
17 import retrofit2.converter.gson.GsonConverterFactory
18 import java.util.concurrent.TimeUnit
19 import javax.inject.Singleton
20
21 @Module
22 @InstallIn(SingletonComponent::class)
23 object NetworkModule {
24
25     private const val TIMEOUT_SECONDS = 20L
26
27     @Provides
28     @Singleton
29     fun provideLoggingInterceptor(): HttpLoggingInterceptor =
30         HttpLoggingInterceptor().apply {
31             // BASIC prints one line per request (method + URL + status): enough to see
32             // in Logcat that only one /auth/login runs. Never log bodies: they hold the password.
33             level = if (BuildConfig.DEBUG) {
34                 HttpLoggingInterceptor.Level.BASIC
35             } else {
36                 HttpLoggingInterceptor.Level.NONE
37             }
38             redactHeader(AuthHeaders.AUTHORIZATION)
39         }
40
41     @Provides
42     @Singleton
43     @PlainClient
44     fun providePlainClient(logging: HttpLoggingInterceptor): OkHttpClient =
45         baseClientBuilder()
46             .addInterceptor(logging)
47             .build()
48
49     @Provides
50     @Singleton
51     @AuthClient
52     fun provideAuthClient(
53         authInterceptor: AuthInterceptor,
54         tokenAuthenticator: TokenAuthenticator,
55         logging: HttpLoggingInterceptor,
56     ): OkHttpClient =
57         baseClientBuilder()

```

```

58         .addInterceptor(authInterceptor) // first: attach a valid token
59         .addInterceptor(logging)        // then: log the final request
60         .authenticator(tokenAuthenticator)
61         .build()
62
63     @Provides
64     @Singleton
65     fun provideAuthApi(@PlainClient client: OkHttpClient): AuthApi =
66         buildRetrofit(client).create(AuthApi::class.java)
67
68     @Provides
69     @Singleton
70     fun provideProductApi(@AuthClient client: OkHttpClient): ProductApi =
71         buildRetrofit(client).create(ProductApi::class.java)
72
73     private fun baseClientBuilder(): OkHttpClient.Builder =
74         OkHttpClient.Builder()
75             .connectTimeout(TIMEOUT_SECONDS, TimeUnit.SECONDS)
76             .readTimeout(TIMEOUT_SECONDS, TimeUnit.SECONDS)
77
78     private fun buildRetrofit(client: OkHttpClient): Retrofit =
79         Retrofit.Builder()
80             .baseUrl(ApiConfig.BASE_URL)
81             .client(client)
82             .addConverterFactory(GsonConverterFactory.create())
83             .build()
84 }

```

الشرح

مصطلحات Hilt في هذا الملف:

المصطلح	المعنى
<code>@Module</code> + <code>@InstallIn(SingletonComponent::class)</code>	وحدة وصفات تعيش طوال عمر التطبيق
<code>@Provides</code>	"هكذا تصنع هذا الشيء". نحتاجه لكلاسات من مكتبات خارجية لا نستطيع وضع <code>@Inject</code> عليها
<code>@Singleton</code>	اصنعه مرة واحدة فقط وأعد استخدامه

`provideLoggingInterceptor`: سجل الطلبات في Logcat.

- مستوى `BASIC`: سطر واحد لكل طلب (الطريقة، والعنوان، والنتيجة). هذا كافٍ لإثبات أن login حدث مرة واحدة.
- يعمل في نسخة التطوير (`DEBUG`) فقط، ومعطّل في النسخة النهائية.
- `redactHeader("Authorization")`: يخفي الـ token من السجل.
- لا نسجّل جسم الطلبات أبداً، لأن جسم طلب login يحتوي كلمة المرور.

`providePlainClient`: العميل العادي. فيه سجل الطلبات فقط، بلا `AuthInterceptor`.

`provideAuthClient`: العميل المحمي:

- الترتيب مهم: `authInterceptor` أولاً ليضيف الـ token، ثم `logging` ليسجّل الطلب النهائي.
- `authenticator(tokenAuthenticator)`: لمعالجة رد 401.

`provideProductApi` و `provideAuthApi`: كل API مع عميله:

- `AuthApi` ← `@PlainClient`.

• `@AuthClient` ← `ProductApi`

الدوال المساعدة:

- `baseClientBuilder`: الإعدادات المشتركة، أي مهلة 20 ثانية (**timeout**) للاتصال والقراءة.
- `buildRetrofit`: ينشئ Retrofit بالعنوان الأساسي ومحوّل Gson.

خلاصة هذا الملف: هنا تحديداً نمنع الحلقة اللانهائية. طلب login لا يمر أبداً بالـ `AuthInterceptor`.

27. `di/DataModule.kt`

الدور: ربط كل interface بتنفيذه الحقيقي.

app/src/main/java/com/example/fakestore/di/DataModule.kt

27 lines

```
1 package com.example.fakestore.di
2
3 import com.example.fakestore.data.auth.Clock
4 import com.example.fakestore.data.auth.EncryptedTokenStorage
5 import com.example.fakestore.data.auth.SystemTimeClock
6 import com.example.fakestore.data.auth.TokenStorage
7 import com.example.fakestore.data.repository.ProductRepository
8 import com.example.fakestore.data.repository.ProductRepositoryImpl
9 import dagger.Binds
10 import dagger.Module
11 import dagger.hilt.InstallIn
12 import dagger.hilt.components.SingletonComponent
13
14 /** Binds each interface to its production implementation. */
15 @Module
16 @InstallIn(SingletonComponent::class)
17 abstract class DataModule {
18
19     @Binds
20     abstract fun bindTokenStorage(impl: EncryptedTokenStorage): TokenStorage
21
22     @Binds
23     abstract fun bindClock(impl: SystemTimeClock): Clock
24
25     @Binds
26     abstract fun bindProductRepository(impl: ProductRepositoryImpl): ProductRepository
27 }
```

الشرح

`@Binds` معناها: "عندما يطلب أحد X أعطه Y". وهي أخف من `@Provides` لأنها لا تحتاج كتابة كود إنشاء.

يُطلب	يُعطى
<code>TokenStorage</code>	<code>EncryptedTokenStorage</code>
<code>Clock</code>	<code>SystemTimeClock</code>
<code>ProductRepository</code>	<code>ProductRepositoryImpl</code>

لتغيير طريقة التخزين لاحقاً (مثلاً إلى DataStore): غيّر سطراً واحداً هنا.

الجزء الخامس: أدوات الواجهة المشتركة

28. `ui/common/UiState.kt`

الدور: حالات أي شاشة تحمّل بيانات.

app/src/main/java/com/example/fakestore/ui/common/UiState.kt

10 lines

```
1 package com.example.fakestore.ui.common
2
3 import androidx.annotation.StringRes
4
5 /** State of a screen that loads data: exactly one of these at a time. */
6 sealed interface UiState<out T> {
7     data object Loading : UiState<Nothing>
8     data class Success<T>(val data: T) : UiState<T>
9     data class Error(@StringRes val messageRes: Int) : UiState<Nothing>
10 }
```

الشرح

- `sealed interface` بثلاث حالات فقط، والشاشة في واحدة منها دائماً:

الحالة	ماذا تعرض الشاشة
<code>Loading</code>	دائرة تحميل
<code>Success(data)</code>	البيانات
<code>Error(messageRes)</code>	رسالة خطأ

- `<out T>`: يسمح باستخدام `Loading` (النوع `Nothing`) مكان أي `<أي شيء> UiState`.
- `@StringRes Int`: رقم مورد نصي مثل `R.string.error_network`، وليس نصاً مكتوباً. هكذا تبقى النصوص في `strings.xml`، فيسهل ترجمتها لاحقاً.

29. `ui/common/ErrorMessage.kt`

الدور: تحويل أي خطأ تقني إلى رسالة مفهومة للمستخدم.

app/src/main/java/com/example/fakestore/ui/common/ErrorMessage.kt

18 lines

```
1 package com.example.fakestore.ui.common
2
3 import androidx.annotation.StringRes
4 import com.example.fakestore.R
5 import com.example.fakestore.data.auth.InvalidCredentialsException
6 import com.example.fakestore.data.auth.SessionExpiredException
7 import retrofit2.HttpException
8 import java.io.IOException
9
10 /** Maps any failure to a message the user can understand. Order matters: most specific first. */
```

```

11 @StringRes
12 fun Throwable.toMessageRes(): Int = when (this) {
13     is InvalidCredentialsException -> R.string.error_invalid_credentials
14     is SessionExpiredException -> R.string.error_session_expired
15     is HttpException -> R.string.error_server
16     is IOException -> R.string.error_network
17     else -> R.string.error_unknown
18 }

```

الشرح

الترتيب مهم: من الأكثر تحديداً إلى الأعم. `InvalidCredentialsException` و `SessionExpiredException` يرثان من `IOException`. لو وضعنا `is IOException` أولاً، لظهرت لهما رسالة "لا يوجد إنترنت" الخاطئة.

الخطأ	الرسالة
<code>InvalidCredentialsException</code>	اسم المستخدم أو كلمة المرور خاطئة
<code>SessionExpiredException</code>	انتهت الجلسة
<code>HttpException</code> (من Retrofit، أي رد بكوند خطأ)	خطأ في السيرفر
<code>IOException</code>	لا يوجد إنترنت
أي شيء آخر	حدث خطأ ما

30. `ui/common/FlowExt.kt`

الدور: دالة مساعدة لمراقبة الـ Flow بأمان داخل Fragment.

app/src/main/java/com/example/fakestore/ui/common/FlowExt.kt

20 lines

```

1 package com.example.fakestore.ui.common
2
3 import androidx.fragment.app.Fragment
4 import androidx.lifecycle.Lifecycle
5 import androidx.lifecycle.LifecycleScope
6 import androidx.lifecycle.repeatOnLifecycle
7 import kotlinx.coroutines.flow.Flow
8 import kotlinx.coroutines.launch
9
10 /**
11  * Collects [flow] only while the fragment's view is visible (STARTED), and stops
12  * automatically when the view is destroyed. Avoids leaks and updates to dead views.
13  */
14 fun <T> Fragment.collectWhenStarted(flow: Flow<T>, action: (T) -> Unit) {
15     viewLifecycleOwner.lifecycleScope.launch {
16         viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
17             flow.collect { action(it) }
18         }
19     }
20 }

```

الشرح

المشكلة: الـ Fragment له دورة حياة، وواجهته (**View**) تُدمّر وتُنشأ من جديد، مثلاً عند الانتقال لشاشة أخرى ثم العودة. لو بقينا نحدّث واجهة مدمّرة لحدث crash أو تسريب ذاكرة.

الحل:

- `viewLifecycleOwner`: دورة حياة **الواجهة** فقط، وليس الـ Fragment كله.
 - `:repeatOnLifecycle(STARTED)`
 - يبدأ المراقبة عندما تصبح الشاشة ظاهرة.
 - يوقفها عندما تختفي، مثلاً عند الذهاب للخلفية.
 - يعيد تشغيلها عند العودة.
 - عند تدمير الواجهة يتوقف كل شيء تلقائياً.
- الاستخدام:** في كل Fragment نكتب سطرًا واحدًا:

```
collectWhenStarted(viewModel.state) { render(it) }
```

31. `ui/common/Formatters.kt`

الدور: تنسيق السعر.

app/src/main/java/com/example/fakestore/ui/common/Formatters.kt

7 lines

```
1 package com.example.fakestore.ui.common
2
3 import java.text.NumberFormat
4 import java.util.Locale
5
6 /** FakeStore prices are in US dollars. */
7 fun Double.formatPrice(): String = NumberFormat.getCurrencyInstance(Locale.US).format(this)
```

الشرح

- `NumberFormat.getCurrencyInstance(Locale.US)`: يحوّل `109.95` إلى `$109.95`.
- `Locale.US` ثابت لأن أسعار FakeStore بالدولار، مهما كانت لغة الجهاز.

الجزء السادس: الشاشات (View + ViewModel)

نمط MVVM في كل شاشة:

الطبقة	الملفات	الدور
View	XML ملف + Fragment	تعرض الحالة، وتُبلغ ال ViewModel بضغوطات المستخدم
ViewModel	...ViewModel.kt	يطلب البيانات، ويحوّلها إلى UiState، ويبقى حياً عند تدوير الشاشة
Model	AuthManager / Repository	البيانات

32. `res/values/strings.xml`

الدور: كل نصوص التطبيق.

```
app/src/main/res/values/strings.xml 33 lines
1 <resources>
2   <string name="app_name">FakeStore</string>
3
4   <!-- Screen titles -->
5   <string name="title_login">Login</string>
6   <string name="title_products">Products</string>
7   <string name="title_product_detail">Product details</string>
8
9   <!-- Login -->
10  <string name="login_heading">Welcome back</string>
11  <string name="login_subtitle">Sign in to browse products</string>
12  <string name="hint_username">Username</string>
13  <string name="hint_password">Password</string>
14  <string name="action_login">Log in</string>
15  <string name="test_account_hint">Test account: mor_2314 / 83r5^</string>
16
17  <!-- Products -->
18  <string name="menu_refresh">Refresh</string>
19  <string name="menu_concurrency_test">Run concurrency test</string>
20  <string name="menu_logout">Log out</string>
21  <string name="concurrency_test_done">%1$d parallel requests finished. Check Logcat: only one
/auth/login.</string>
22  <string name="product_rating">★ %1$.1f (%2$d reviews)</string>
23  <string name="product_image">Product image</string>
24
25  <!-- Errors -->
26  <string name="retry">Retry</string>
27  <string name="error_empty_fields">Please enter your username and password.</string>
28  <string name="error_invalid_credentials">Invalid username or password.</string>
29  <string name="error_session_expired">Your session has expired. Please log in again.</string>
30  <string name="error_server">The server returned an error. Please try again.</string>
31  <string name="error_network">No internet connection. Check your network and try again.</string>
32  <string name="error_unknown">Something went wrong. Please try again.</string>
33 </resources>
```

الشرح

- لا نكتب نصوصاً داخل الكود أو XML مباشرة، فكل نص هنا. هذا يسهّل الترجمة لاحقاً بملف `values-ar/strings.xml`.

- `%1$d` و `%1$.1f`: أماكن لقيم تُملأ وقت التشغيل:
 - `%1$d`: رقم صحيح (الأول).
 - `%1$.1f`: رقم عشري بخانة واحدة بعد الفاصلة.
- `test_account_hint`: يعرض بيانات حساب الاختبار على شاشة الدخول، لتسهيل التجربة على المراجع.

33. `res/values/themes.xml`

الدور: تيم التطبيق.

app/src/main/res/values/themes.xml

3 lines

```
1 <resources>
2   <style name="Theme.FakeStore" parent="Theme.Material3.DayNight.NoActionBar" />
3 </resources>
```

الشرح

- `Theme.Material3.DayNight`: تصميم Material 3 مع دعم الوضع الليلي تلقائياً.
- `NoActionBar`: بلا شريط علوي افتراضي، لأننا نضع `MaterialToolbar` بأنفسنا في `activity_main.xml`.

34. `res/layout/activity_main.xml`

الدور: هيكل الشاشة الرئيسية: شريط علوي، ومساحة تتبدل فيها الشاشات.

app/src/main/res/layout/activity_main.xml

26 lines

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3   xmlns:app="http://schemas.android.com/apk/res-auto"
4   android:layout_width="match_parent"
5   android:layout_height="match_parent"
6   android:orientation="vertical">
7
8   <com.google.android.material.appbar.AppBarLayout
9     android:layout_width="match_parent"
10    android:layout_height="wrap_content">
11
12     <com.google.android.material.appbar.MaterialToolbar
13       android:id="@+id/toolbar"
14       android:layout_width="match_parent"
15       android:layout_height="?attr/actionBarSize" />
16   </com.google.android.material.appbar.AppBarLayout>
17
18   <!-- The graph is set from code (MainActivity) to pick the start screen. -->
19   <androidx.fragment.app.FragmentContainerView
20     android:id="@+id/nav_host"
21     android:name="androidx.navigation.fragment.NavHostFragment"
22     android:layout_width="match_parent"
23     android:layout_height="0dp"
24     android:layout_weight="1"
25     app:defaultNavHost="true" />
26 </LinearLayout>
```

الشرح

- `MaterialToolbar` داخل `AppBarLayout`: الشريط العلوي. يعرض عنوان الشاشة الحالية، وسهم الرجوع، والقائمة (:).
- `FragmentContainerView` مع `NavHostFragment`: **المساحة التي تُعرض فيها الشاشات (Fragments)**، ومكوّن `Navigation` يبدّلها.
- `app:defaultNavHost="true"`: زر الرجوع في الجهاز يتحكم بالتنقل داخل هذه المساحة.
- `layout_height="0dp"` مع `layout_weight="1"`: تأخذ كل المساحة المتبقية تحت الشريط.
- **لا يوجد** `app:navGraph` **هنا عن قصد**. نحدد خريطة التنقل من الكود في `MainActivity`، لنختار شاشة البداية.

35. `res/navigation/nav_graph.xml`

الدور: خريطة التنقل: ما الشاشات، وكيف نتنقل بينها.

app/src/main/res/navigation/nav_graph.xml

47 lines

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <!--
3     Screens and the transitions between them. The real start destination is chosen in
4     MainActivity (login or products, depending on whether a session exists).
5 -->
6 <navigation xmlns:android="http://schemas.android.com/apk/res/android"
7     xmlns:app="http://schemas.android.com/apk/res-auto"
8     android:id="@+id/nav_graph"
9     app:startDestination="@id/loginFragment">
10
11     <fragment
12         android:id="@+id/loginFragment"
13         android:name="com.example.fakestore.ui.login.LoginFragment"
14         android:label="@string/title_login">
15         <!-- popUpTo login (inclusive): Back from Products must not return to Login. -->
16         <action
17             android:id="@+id/action_login_to_products"
18             app:destination="@id/productsFragment"
19             app:popUpTo="@id/loginFragment"
20             app:popUpToInclusive="true" />
21     </fragment>
22
23     <fragment
24         android:id="@+id/productsFragment"
25         android:name="com.example.fakestore.ui.products.ProductsFragment"
26         android:label="@string/title_products">
27         <action
28             android:id="@+id/action_products_to_detail"
29             app:destination="@id/productDetailFragment" />
30     </fragment>
31
32     <fragment
33         android:id="@+id/productDetailFragment"
34         android:name="com.example.fakestore.ui.detail.ProductDetailFragment"
35         android:label="@string/title_product_detail">
36         <argument
37             android:name="productId"
38             app:argType="integer" />
39     </fragment>
40
41     <!-- Logout from anywhere: clear the whole back stack, then show Login. -->
42     <action
43         android:id="@+id/action_global_login"
```

```

44         app:destination="@id/loginFragment"
45         app:popUpTo="@id/nav_graph"
46         app:popUpToInclusive="true" />
47     </navigation>

```

الشرح

- `<fragment>`: كل شاشة لها `id` واسم كلاسها (`android:name`) وعنوانها (`label`)، والعنوان يظهر تلقائياً في الشريط العلوي.
- `<action>`: انتقال مسموح من شاشة إلى أخرى.
- `: action_login_to_products`
- `popUpTo="@id/loginFragment" + popUpToInclusive="true"`: عند الانتقال للمنتجات تُحذف شاشة Login من سجل الرجوع (back stack).
- لولا ذلك، لعاد المستخدم لشاشة Login بزر الرجوع وهو مسجل الدخول.
- `<argument android:name="productId">`
- شاشة التفاصيل تحتاج رقم المنتج.
- `argType="integer"`: نوعه رقم صحيح.
- `action_global_login`: انتقال عام (global action)، يمكن استخدامه من أي شاشة:
- `popUpTo="@id/nav_graph" + inclusive`: **يمسح كل الشاشات** ثم يفتح Login.
- يُستخدم في Logout، وفي الخروج الإجباري. هذا تحقيق لمتطلب *"Redirect user to Login screen"*. بعده، زر الرجوع يُغلق التطبيق بدل العودة للمنتجات.

36. MainActivity.kt

الدور: ال Activity الوحيدة. تختار شاشة البداية، وتستمتع للخروج الإجباري.

app/src/main/java/com/example/fakestore/MainActivity.kt

85 lines

```

1  package com.example.fakestore
2
3  import android.os.Bundle
4  import android.widget.Toast
5  import androidx.appcompat.app.AppCompatActivity
6  import androidx.lifecycle.Lifecycle
7  import androidx.lifecycle.LifecycleScope
8  import androidx.lifecycle.RepeatOnLifecycle
9  import androidx.navigation.NavController
10 import androidx.navigation.fragment.NavHostFragment
11 import androidx.navigation.ui.AppBarConfiguration
12 import androidx.navigation.ui.navigateUp
13 import androidx.navigation.ui.setupActionBarWithNavController
14 import com.example.fakestore.data.auth.AuthEvent
15 import com.example.fakestore.data.auth.AuthManager
16 import com.example.fakestore.databinding.ActivityMainBinding
17 import dagger.hilt.android.AndroidEntryPoint
18 import kotlinx.coroutines.launch
19 import javax.inject.Inject
20
21 /**
22  * Single activity hosting all screens (fragments). It also listens for forced logouts,

```

```

23  * because they can happen on any screen.
24  */
25  @AndroidEntryPoint
26  class MainActivity : AppCompatActivity() {
27
28      @Inject
29      lateinit var authManager: AuthManager
30
31      private lateinit var binding: ActivityMainBinding
32      private lateinit var navController: NavController
33      private lateinit var appBarConfiguration: AppBarConfiguration
34
35      override fun onCreate(savedInstanceState: Bundle?) {
36          super.onCreate(savedInstanceState)
37          binding = ActivityMainBinding.inflate(layoutInflater)
38          setContentView(binding.root)
39          setSupportActionBar(binding.toolbar)
40
41          val navHost = supportFragmentManager.findFragmentById(R.id.nav_host) as NavHostFragment
42          navController = navHost.navController
43
44          // Start on the product list if a session exists (an expired token is refreshed
45          // automatically on the first request), otherwise on the login screen.
46          val graph = navController.navInflater.inflate(R.navigation.nav_graph).apply {
47              setStartDestination(
48                  if (authManager.isLoggedIn()) R.id.productsFragment else R.id.loginFragment,
49              )
50          }
51          navController.setGraph(graph, null)
52
53          // Login and Products are top-level: no "back" arrow on them.
54          appBarConfiguration = AppBarConfiguration(setOf(R.id.loginFragment, R.id.productsFragment))
55          setupActionBarWithNavController(navController, appBarConfiguration)
56
57          observeSession()
58      }
59
60      override fun onSupportNavigateUp(): Boolean =
61          navController.navigateUp(appBarConfiguration) || super.onSupportNavigateUp()
62
63      private fun observeSession() {
64          lifecycleScope.launch {
65              repeatOnLifecycle(Lifecycle.State.STARTED) {
66                  // The session may have been cleared while the app was in the background.
67                  if (!authManager.isLoggedIn()) goToLogin(showMessage = false)
68
69                  authManager.events.collect { event ->
70                      when (event) {
71                          AuthEvent.SessionExpired -> goToLogin(showMessage = true)
72                      }
73                  }
74              }
75          }
76      }
77
78      private fun goToLogin(showMessage: Boolean) {
79          if (navController.currentDestination?.id == R.id.loginFragment) return
80          if (showMessage) {
81              Toast.makeText(this, R.string.error_session_expired, Toast.LENGTH_LONG).show()
82          }
83          navController.navigate(R.id.action_global_login)
84      }
85  }

```

الشرح

`@AndroidEntryPoint`: يسمح لـ Hilt بحقن الاعتماديات هنا وفي الـ Fragments داخلها.

`@Inject lateinit var authManager`: حقن في حقل (**field injection**). Hilt يملؤه قبل `onCreate`. نستخدم هذا الأسلوب لأن Android هو من ينشئ الـ Activity، وليس نحن، فلا نستطيع تمرير شيء لـ `constructor`.

`: onCreate`

1. `ActivityMainBinding.inflate`: ننشئ الواجهة عبر `ViewBinding`.

2. `setSupportActionBar(binding.toolbar)`: نجعل الـ Toolbar هو الشريط العلوي.

3. نحصل على `navController` من `NavHostFragment`.

4. **اختيار شاشة البداية**: - نحمل خريطة التنقل من XML. - نحدد البداية: إذا كانت هناك جلسة محفوظة نبدأ بـ **المنتجات**، وإلا بـ **Login**. - حتى لو كان الـ token منتهياً، سيُجدد تلقائياً مع أول طلب.

5. `AppBarConfiguration(setOf(login, products))`: هاتان شاشتان رئيسيتان (**top-level**)، فلا يظهر عليهما سهم الرجوع. أما التفاصيل فيظهر عليها.

`observeSession()`: مراقبة الجلسة، تعمل فقط والتطبيق ظاهر.

1. **أولاً**: إذا لم تكن هناك جلسة نذهب لشاشة Login. قد تكون الجلسة مُسحت والتطبيق في الخلفية، حين لم يكن أحد يستمع للحدث.

2. **ثانياً**: نستمع لـ `authManager.events`. عند وصول `SessionExpired` ننقل لـ Login مع رسالة.

لماذا هنا في MainActivity وليس في كل Fragment؟ الخروج الإجباري قد يحدث في أي شاشة، أثناء القائمة أو التفاصيل. الـ Activity موجودة دائماً، فهي المكان المناسب لاستقبال الحدث مرة واحدة.

`goToLogin`: إذا كنا أصلاً في Login لا نفعل شيئاً، فهذا يمنع فتح Login مرتين.

37. `ui/login/LoginViewModel.kt`

الدور: منطق شاشة تسجيل الدخول.

app/src/main/java/com/example/fakestore/ui/login/LoginViewModel.kt

53 lines

```
1 package com.example.fakestore.ui.login
2
3 import androidx.annotation.StringRes
4 import androidx.lifecycle.ViewModel
5 import androidx.lifecycle.ViewModelScope
6 import com.example.fakestore.R
7 import com.example.fakestore.data.auth.AuthManager
8 import com.example.fakestore.ui.common.toMessageRes
9 import dagger.hilt.android.lifecycle.HiltViewModel
10 import kotlinx.coroutines.CancellationException
11 import kotlinx.coroutines.flow.MutableStateFlow
12 import kotlinx.coroutines.flow.StateFlow
13 import kotlinx.coroutines.flow.asStateFlow
14 import kotlinx.coroutines.flow.update
15 import kotlinx.coroutines.launch
16 import javax.inject.Inject
17
18 data class LoginUiState(
19     val isLoading: Boolean = false,
20     @StringRes val errorRes: Int? = null,
```

```

21     val isLoggedIn: Boolean = false,
22 )
23
24 @HiltViewModel
25 class LoginViewModel @Inject constructor(
26     private val authManager: AuthManager,
27 ) : ViewModel() {
28
29     private val _state = MutableStateFlow(LoginUiState())
30     val state: StateFlow<LoginUiState> = _state.asStateFlow()
31
32     fun login(username: String, password: String) {
33         if (_state.value.isLoading) return // ignore double taps
34
35         val trimmedUsername = username.trim()
36         if (trimmedUsername.isEmpty() || password.isEmpty()) {
37             _state.update { it.copy(errorRes = R.string.error_empty_fields) }
38             return
39         }
40
41         viewModelScope.launch {
42             _state.update { it.copy(isLoading = true, errorRes = null) }
43             try {
44                 authManager.login(trimmedUsername, password)
45                 _state.update { it.copy(isLoading = false, isLoggedIn = true) }
46             } catch (e: CancellationException) {
47                 throw e
48             } catch (e: Exception) {
49                 _state.update { it.copy(isLoading = false, errorRes = e.toMessageRes()) }
50             }
51         }
52     }
53 }

```

الشرح

`LoginUiState` : كل ما تحتاجه الشاشة في كائن واحد:

الحقل	المعنى
<code>isLoading</code>	يجري تسجيل الدخول؟ (دائرة تحميل، وزر معطل)
<code>errorRes</code>	رسالة خطأ، أو <code>null</code> إن لم يكن هناك خطأ
<code>isLoggedIn</code>	نجح الدخول؟ (إشارة للانتقال)

`StateFlow` / `MutableStateFlow` :

- `_state` خاص وقابل للتعديل.
- `state` عام وللقراءة فقط: **الشاشة تقرأ ولا تعدّل**.
- `_state.update { it.copy(...) }` : ينسخ الحالة الحالية مع تغيير حقول معينة، بشكل آمن مع الـ `threads`.

خطوة بخطوة: `login()`

1. إذا كان التحميل جارياً تجاهل الضغطة، فهذا يمنع **الضغط المزدوج** على الزر.
2. التحقق المحلي: إذا كانت الحقول فارغة نعرض خطأ **دون** إرسال طلب.
3. `viewModelScope.launch` : نبدأ `coroutine` تُلغى تلقائياً إذا أُغلقت الشاشة نهائياً.
4. نجعل الحالة `Loading`.

5. نستدعي `authManager.login()` - **نجاح**: `isLoggedIn = true` - **فشل**: نحول الخطأ إلى رسالة عبر `toMessageRes()`.

```
:catch (e: CancellationCancellation) { throw e }
```

- إلغاء الـ `coroutine` ليس خطأً.
- إذا التقطناه ضمن `catch (e: Exception)` ولم نمرره، تتعطل آلية الإلغاء.
- لذلك نعيد رميه دائماً، وهذه ممارسة احترافية مهمة.

38. `res/layout/fragment_login.xml`

الدور: تصميم شاشة تسجيل الدخول.

app/src/main/res/layout/fragment_login.xml

97 lines

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:fillViewport="true">
7
8      <LinearLayout
9          android:layout_width="match_parent"
10         android:layout_height="wrap_content"
11         android:layout_gravity="center_vertical"
12         android:orientation="vertical"
13         android:padding="24dp">
14
15         <TextView
16             android:layout_width="wrap_content"
17             android:layout_height="wrap_content"
18             android:text="@string/login_heading"
19             android:textAppearance="?attr/textAppearanceHeadlineMedium" />
20
21         <TextView
22             android:layout_width="wrap_content"
23             android:layout_height="wrap_content"
24             android:layout_marginTop="4dp"
25             android:layout_marginBottom="24dp"
26             android:text="@string/login_subtitle"
27             android:textAppearance="?attr/textAppearanceBodyMedium" />
28
29         <com.google.android.material.textfield.TextInputLayout
30             android:id="@+id/username_layout"
31             style="@style/Widget.Material3.TextInputLayout.OutlinedBox"
32             android:layout_width="match_parent"
33             android:layout_height="wrap_content"
34             android:hint="@string/hint_username">
35
36             <com.google.android.material.textfield.TextInputEditText
37                 android:id="@+id/username_input"
38                 android:layout_width="match_parent"
39                 android:layout_height="wrap_content"
40                 android:autofillHints="username"
41                 android:imeOptions="actionNext"
42                 android:inputType="text"
43                 android:maxLines="1" />
44         </com.google.android.material.textfield.TextInputLayout>
45
46         <com.google.android.material.textfield.TextInputLayout
```

```

47         android:id="@+id/password_layout"
48         style="@style/Widget.Material3.TextInputLayout.OutlinedBox"
49         android:layout_width="match_parent"
50         android:layout_height="wrap_content"
51         android:layout_marginTop="12dp"
52         android:hint="@string/hint_password"
53         app:endIconMode="password_toggle">
54
55         <com.google.android.material.textfield.TextInputEditText
56             android:id="@+id/password_input"
57             android:layout_width="match_parent"
58             android:layout_height="wrap_content"
59             android:autofillHints="password"
60             android:imeOptions="actionDone"
61             android:inputType="textPassword"
62             android:maxLines="1" />
63     </com.google.android.material.textfield.TextInputLayout>
64
65     <TextView
66         android:id="@+id/error_text"
67         android:layout_width="match_parent"
68         android:layout_height="wrap_content"
69         android:layout_marginTop="12dp"
70         android:textColor="?attr/colorError"
71         android:visibility="gone" />
72
73     <com.google.android.material.button.MaterialButton
74         android:id="@+id/login_button"
75         android:layout_width="match_parent"
76         android:layout_height="wrap_content"
77         android:layout_marginTop="20dp"
78         android:text="@string/action_login" />
79
80     <com.google.android.material.progressindicator.CircularProgressIndicator
81         android:id="@+id/progress"
82         android:layout_width="wrap_content"
83         android:layout_height="wrap_content"
84         android:layout_gravity="center_horizontal"
85         android:layout_marginTop="16dp"
86         android:indeterminate="true"
87         android:visibility="gone" />
88
89     <TextView
90         android:layout_width="wrap_content"
91         android:layout_height="wrap_content"
92         android:layout_gravity="center_horizontal"
93         android:layout_marginTop="24dp"
94         android:text="@string/test_account_hint"
95         android:textAppearance="?attr/textAppearanceBodySmall" />
96 </LinearLayout>
97 </ScrollView>

```

الشرح

- مع `ScrollView` `fillViewport`: الشاشة قابلة للتمرير إذا غطتها لوحة المفاتيح.
- `TextInputLayout` + `TextInputEditText`: حقل إدخال من Material، فيه عنوان عائم وإطار.
- `inputType="textPassword"`: يُخفي كلمة المرور بنقاط.
- `endIconMode="password_toggle"`: أيقونة عين لإظهار كلمة المرور أو إخفائها.
- `imeOptions`: زر لوحة المفاتيح:
 - `actionNext` في حقل الاسم: ينتقل لحقل كلمة المرور.

- `actionDone` في حقل كلمة المرور: ينقذ الدخول.
- `autofillHints`: يسمح لمدير كلمات المرور بملء الحقول.
- `error_text`: رسالة الخطأ، مخفية (`gone`) حتى يحدث خطأ. `?attr/colorError` يجعل لونها أحمر من الريم.
- `CircularProgressIndicator`: دائرة التحميل، مخفية في البداية.

39. `ui/login/LoginFragment.kt`

الدور: شاشة تسجيل الدخول (ال View).

app/src/main/java/com/example/fakestore/ui/login/LoginFragment.kt

66 lines

```

1  package com.example.fakestore.ui.login
2
3  import android.os.Bundle
4  import android.view.View
5  import android.view.inputmethod.EditorInfo
6  import androidx.core.view.isVisible
7  import androidx.fragment.app.Fragment
8  import androidx.fragment.app.viewModels
9  import androidx.navigation.fragment.findNavController
10 import com.example.fakestore.R
11 import com.example.fakestore.databinding.FragmentLoginBinding
12 import com.example.fakestore.ui.common.collectWhenStarted
13 import dagger.hilt.android.AndroidEntryPoint
14
15 @AndroidEntryPoint
16 class LoginFragment : Fragment(R.layout.fragment_login) {
17
18     private val viewModel: LoginViewModel by viewModels()
19
20     private var _binding: FragmentLoginBinding? = null
21     private val binding get() = _binding!!
22
23     override fun onCreateView(view: View, savedInstanceState: Bundle?) {
24         super.onCreateView(view, savedInstanceState)
25         _binding = FragmentLoginBinding.bind(view)
26
27         binding.loginButton.setOnClickListener { submit() }
28         binding.passwordInput.setOnEditorActionListener { _, actionId, _ ->
29             if (actionId == EditorInfo.IME_ACTION_DONE) {
30                 submit()
31                 true
32             } else {
33                 false
34             }
35         }
36
37         collectWhenStarted(viewModel.state) { render(it) }
38     }
39
40     private fun submit() {
41         viewModel.login(
42             username = binding.usernameInput.text?.toString().orEmpty(),
43             password = binding.passwordInput.text?.toString().orEmpty(),
44         )
45     }
46
47     private fun render(state: LoginUiState) {
48         binding.progress.isVisible = state.isLoading

```

```

49         binding.loginButton.isEnabled = !state.isLoading
50         binding.usernameLayout.isEnabled = !state.isLoading
51         binding.passwordLayout.isEnabled = !state.isLoading
52
53         binding.errorText.isVisible = state.errorRes != null
54         state.errorRes?.let { binding.errorText.setText(it) }
55
56         val navController = findNavController()
57         if (state.isLoggedIn && navController.currentDestination?.id == R.id.loginFragment) {
58             navController.navigate(R.id.action_login_to_products)
59         }
60     }
61
62     override fun onDestroyView() {
63         super.onDestroyView()
64         _binding = null
65     }
66 }

```

الشرح

- الـ `Fragment(R.layout.fragment_login)` ينشئ واجهته من هذا الملف تلقائياً.
- `by viewModels()` يحصل على الـ `ViewModel`، و `Hilt` يوفّر له `AuthManager`. الـ `ViewModel` نفسه يعود بعد تدوير الشاشة.

نمط `_binding` / `binding`:

- واجهة الـ `Fragment` تُدمّر قبل الـ `Fragment` نفسه.
- لذلك نجعل `_binding = null` في `onDestroyView`، حتى لا نحتفظ بواجهة ميتة في الذاكرة (**memory leak**).
- `binding` (مع `!!`) للاستخدام المريح بينهما.

`onViewCreated`:

- ربط الزر وزر `Done` في لوحة المفاتيح بالدالة `submit()`.
- `collectWhenStarted(viewModel.state) { render(it) }`: كلما تغيّرت الحالة تُرسم الشاشة من جديد.

`render(state)`: الشاشة **تعكس** الحالة فقط:

- التحميل: تظهر الدائرة، ويُعطّل الزر والحقول.
- الخطأ: تظهر الرسالة.
- `isLoggedIn`: تنتقل للمنتجات.
- الشرط `currentDestination == loginFragment` يمنع تكرار الانتقال، وهو ما كان سيسبب `crash`.

40. `res/layout/view_error.xml`

الدور: واجهة خطأ قابلة لإعادة الاستخدام: رسالة + زر إعادة المحاولة.

app/src/main/res/layout/view_error.xml

24 lines

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <!-- Reusable error state: message + Retry button. Included in the products and detail screens. -->
3 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
4     android:layout_width="match_parent"
5     android:layout_height="wrap_content"

```

```

6     android:layout_gravity="center"
7     android:gravity="center"
8     android:orientation="vertical"
9     android:padding="24dp">
10
11     <TextView
12         android:id="@+id/error_message"
13         android:layout_width="wrap_content"
14         android:layout_height="wrap_content"
15         android:gravity="center"
16         android:textAppearance="?attr/textAppearanceBodyLarge" />
17
18     <com.google.android.material.button.MaterialButton
19         android:id="@+id/retry_button"
20         android:layout_width="wrap_content"
21         android:layout_height="wrap_content"
22         android:layout_marginTop="16dp"
23         android:text="@string/retry" />
24 </LinearLayout>

```

الشرح

- تُضمَّن في شاشتي المنتجات والتفاصيل بالوسم `<include>` ، بدل تكرار نفس التصميم.
- `ViewBinding` يولّد لها `ViewErrorBinding` ، فنصل لعناصرها مثلاً عبر `binding.errorView.retryButton`.

.41 `ui/products/ProductsViewModel.kt`

الدور: منطق شاشة المنتجات.

app/src/main/java/com/example/fakestore/ui/products/ProductsViewModel.kt

92 lines

```

1  package com.example.fakestore.ui.products
2
3  import androidx.annotation.StringRes
4  import androidx.lifecycle.ViewModel
5  import androidx.lifecycle.viewModelScope
6  import com.example.fakestore.data.auth.AuthManager
7  import com.example.fakestore.data.model.Product
8  import com.example.fakestore.data.repository.ProductRepository
9  import com.example.fakestore.ui.common.UiState
10 import com.example.fakestore.ui.common.toMessageRes
11 import dagger.hilt.android.lifecycle.HiltViewModel
12 import kotlinx.coroutines.CancellationException
13 import kotlinx.coroutines.async
14 import kotlinx.coroutines.awaitAll
15 import kotlinx.coroutines.channels.Channel
16 import kotlinx.coroutines.coroutineScope
17 import kotlinx.coroutines.flow.Flow
18 import kotlinx.coroutines.flow.MutableStateFlow
19 import kotlinx.coroutines.flow.StateFlow
20 import kotlinx.coroutines.flow.asStateFlow
21 import kotlinx.coroutines.flow.receiveAsFlow
22 import kotlinx.coroutines.launch
23 import javax.inject.Inject
24
25 /** One-time events for the products screen (navigation, snickers). */
26 sealed interface ProductsEvent {
27     data object LoggedOut : ProductsEvent
28     data class ConcurrencyTestFinished(val requestCount: Int) : ProductsEvent
29     data class ShowError(@StringRes val messageRes: Int) : ProductsEvent

```

```

30 }
31
32 @HiltViewModel
33 class ProductsViewModel @Inject constructor(
34     private val repository: ProductRepository,
35     private val authManager: AuthManager,
36 ) : ViewModel() {
37
38     private val _state = MutableStateFlow<UiState<List<Product>>>(UiState.Loading)
39     val state: StateFlow<UiState<List<Product>>> = _state.asStateFlow()
40
41     // Channel: each event is delivered once, even if the screen is rotated.
42     private val _events = Channel<ProductsEvent>(Channel.BUFFERED)
43     val events: Flow<ProductsEvent> = _events.receiveAsFlow()
44
45     init {
46         loadProducts()
47     }
48
49     fun loadProducts() {
50         viewModelScope.launch {
51             _state.value = UiState.Loading
52             _state.value = try {
53                 UiState.Success(repository.getProducts())
54             } catch (e: CancellationException) {
55                 throw e
56             } catch (e: Exception) {
57                 UiState.Error(e.toMessageRes())
58             }
59         }
60     }
61
62     /**
63      * Demo for the reviewer: fires [CONCURRENT_REQUESTS] product requests at the same time.
64      * Wait > 60 s after login, run it, and Logcat shows a single `/auth/login`.
65      */
66     fun runConcurrencyTest() {
67         viewModelScope.launch {
68             val event = try {
69                 coroutineScope {
70                     (1..CONCURRENT_REQUESTS)
71                     .map { id -> async { repository.getProduct(id) } }
72                     .awaitAll()
73                 }
74                 ProductsEvent.ConcurrencyTestFinished(CONCURRENT_REQUESTS)
75             } catch (e: CancellationException) {
76                 throw e
77             } catch (e: Exception) {
78                 ProductsEvent.ShowError(e.toMessageRes())
79             }
80             _events.send(event)
81         }
82     }
83
84     fun logout() {
85         authManager.logout()
86         viewModelScope.launch { _events.send(ProductsEvent.LoggedOut) }
87     }
88
89     private companion object {
90         const val CONCURRENT_REQUESTS = 5
91     }
92 }

```

الشرح

`ProductsEvent` : أحداث لمرة واحدة:

- `LoggedIn` : الانتقال لـ `Login`.
- `ConcurrencyTestFinished` : عرض رسالة نجاح اختبار التزامن.
- `ShowError` : عرض رسالة خطأ.

الحالة مقابل الأحداث:

النوع	السلوك
<code>state</code>	<code>StateFlow</code> يحتفظ بآخر قيمة. القائمة تبقى بعد تدوير الشاشة
<code>events</code>	<code>Channel</code> كل حدث يُستهلك مرة واحدة. رسالة الخروج لا تتكرر عند التدوير

`init { loadProducts() }` : يبدأ التحميل فور إنشاء الـ `ViewModel`، مرة واحدة فقط وليس عند كل تدوير.

`loadProducts()` : `Loading`، ثم `Success` أو `Error`.

- الطلب يمر تلقائياً بالـ `AuthInterceptor`، فلا يوجد أي ذكر للـ `token` هنا.
- هذا هدف التصميم كله: الـ `ViewModel` لا يعرف شيئاً عن الـ `tokens`.

★ `runConcurrencyTest()` : أداة لإثبات متطلب التزامن للمراجع

- `coroutineScope { ... }` : ينتظر انتهاء كل الطلبات بداخله.
- `async { repository.getProduct(id) }` : يطلق طلباً دون انتظار انتهائه، فتنتقل الطلبات الخمسة في نفس اللحظة.
- `awaitAll()` : ينتظر الخمسة معاً.
- طريقة العرض: سجّل الدخول، وانتظر أكثر من 60 ثانية، ثم اضغط "Run concurrency test". في `Logcat` سيظهر طلب `login` واحد فقط ثم 5 طلبات منتجات.
- `logout()` : يسمح الجلسة، ثم يرسل حدث `LoggedIn` لنتنقل الشاشة إلى `Login`.

42. `res/layout/item_product.xml`

الدور: تصميم صف واحد في القائمة.

```
app/src/main/res/layout/item_product.xml 57 lines

1 <?xml version="1.0" encoding="utf-8"?>
2 <!-- One row of the RecyclerView. Rows are recycled: the same views show different products. -->
3 <com.google.android.material.card.MaterialCardView
xmlns:android="http://schemas.android.com/apk/res/android"
4     android:layout_width="match_parent"
5     android:layout_height="wrap_content"
6     android:layout_margin="6dp"
7     android:clickable="true"
8     android:focusable="true">
9
10    <LinearLayout
11        android:layout_width="match_parent"
12        android:layout_height="wrap_content"
13        android:gravity="center_vertical"
14        android:orientation="horizontal"
```

```

15     android:padding="12dp">
16
17     <ImageView
18         android:id="@+id/image"
19         android:layout_width="72dp"
20         android:layout_height="72dp"
21         android:background="@android:color/white"
22         android:contentDescription="@string/product_image"
23         android:padding="4dp"
24         android:scaleType="fitCenter" />
25
26     <LinearLayout
27         android:layout_width="0dp"
28         android:layout_height="wrap_content"
29         android:layout_marginStart="12dp"
30         android:layout_weight="1"
31         android:orientation="vertical">
32
33         <TextView
34             android:id="@+id/title"
35             android:layout_width="match_parent"
36             android:layout_height="wrap_content"
37             android:ellipsize="end"
38             android:maxLines="2"
39             android:textAppearance="?attr/textAppearanceTitleSmall" />
40
41         <TextView
42             android:id="@+id/category"
43             android:layout_width="match_parent"
44             android:layout_height="wrap_content"
45             android:layout_marginTop="2dp"
46             android:textAppearance="?attr/textAppearanceBodySmall" />
47
48         <TextView
49             android:id="@+id/price"
50             android:layout_width="wrap_content"
51             android:layout_height="wrap_content"
52             android:layout_marginTop="4dp"
53             android:textAppearance="?attr/textAppearanceTitleMedium"
54             android:textColor="?attr/colorPrimary" />
55     </LinearLayout>
56 </LinearLayout>
57 </com.google.android.material.card.MaterialCardView>

```

الشرح

- `MaterialCardView`: بطاقة لها ظل وزوايا دائرية، وقابلة للضغط (`clickable`).
- **الصف أفقي**: صورة `72dp`، ثم عمود نصوص (العنوان، والتصنيف، والسعر).
- `layout_weight="1"` + `layout_width="0dp"`: عمود النصوص يأخذ كل المساحة المتبقية بجانب الصورة.
- `ellipsize="end"` + `maxLines="2"`: العنوان الطويل يُقصر بـ "...".

[ui/products/ProductAdapter.kt](#) ★ **43.**

الدور: يربط قائمة المنتجات بالـ `RecyclerView`. هذا يتطلب `"RecyclerView"`.

```

1  package com.example.fakestore.ui.products
2
3  import android.view.LayoutInflater
4  import android.view.ViewGroup
5  import androidx.recyclerview.widget.DiffUtil
6  import androidx.recyclerview.widget.ListAdapter
7  import androidx.recyclerview.widget.RecyclerView
8  import coil.load
9  import com.example.fakestore.data.model.Product
10 import com.example.fakestore.databinding.ItemProductBinding
11 import com.example.fakestore.ui.common.formatPrice
12
13 /**
14  * RecyclerView adapter. [ListAdapter] + [DiffUtil] compute what changed between two lists
15  * on a background thread and update only those rows.
16  */
17 class ProductAdapter(
18     private val onProductClick: (Product) -> Unit,
19 ) : ListAdapter<Product, ProductAdapter.ProductViewHolder>(ProductDiff) {
20
21     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ProductViewHolder {
22         val binding = ItemProductBinding.inflate(LayoutInflater.from(parent.context), parent, false)
23         return ProductViewHolder(binding, onProductClick)
24     }
25
26     override fun onBindViewHolder(holder: ProductViewHolder, position: Int) {
27         holder.bind(getItem(position))
28     }
29
30     /** Holds the views of one row so they can be recycled for other products while scrolling. */
31     class ProductViewHolder(
32         private val binding: ItemProductBinding,
33         onProductClick: (Product) -> Unit,
34     ) : RecyclerView.ViewHolder(binding.root) {
35
36         private var product: Product? = null
37
38         init {
39             // Set the listener once per holder, not on every bind.
40             binding.root.setOnClickListener { product?.let(onProductClick) }
41         }
42
43         fun bind(product: Product) {
44             this.product = product
45             binding.title.text = product.title
46             binding.category.text = product.category
47             binding.price.text = product.price.formatPrice()
48             binding.image.load(product.imageUrl) { crossfade(true) }
49         }
50     }
51
52     private object ProductDiff : DiffUtil.ItemCallback<Product>() {
53         override fun areItemsTheSame(oldItem: Product, newItem: Product) = oldItem.id == newItem.id
54         override fun areContentsTheSame(oldItem: Product, newItem: Product) = oldItem == newItem
55     }
56 }

```

الشرح

كيف يعمل RecyclerView؟ القائمة فيها 20 منتجاً والشاشة تعرض 7 فقط. RecyclerView ينشئ نحو 9 صفوف فقط. عند التمرير، الصف الذي يخرج من الأعلى يُعاد استخدامه (recycled) لعرض منتج جديد في الأسفل. هذا يوفر الذاكرة ويجعل التمرير

`: onCreateViewHolder`

- يُنشئ صفًا جديدًا من `item_product.xml`.
- يُستدعى **قليلاً**، بعدد الصفوف الظاهرة تقريباً.

`: onBindViewHolder`

- يضع بيانات منتج على صف موجود.
- يُستدعى **كثيراً**، مع كل تمرير.

`ProductViewHolder`: يمكّن عناصر الصف (عبر binding) لإعادة استخدامها.

- **مستمع الضغط يُضبط مرة واحدة** في `init`، وليس في كل `bind`، فهذا أكفأ.
- يقرأ المنتج الحالي من المتغير `product`.
- `binding.image.load(url)`: مكتبة **Coil** تحمّل الصورة من الإنترنت في الخلفية، وتخزّنها مؤقتاً (**cache**)، وتعرضها بتأثير تلاشي (`crossfade`).

`: DiffUtil + ListAdapter`

- عند وصول قائمة جديدة عبر `submitList`، يقارن `DiffUtil` القديمة بالجديدة في **الخلفية**، ويحدّث فقط الصفوف التي تغيّرت، مع حركة.
- `areItemsTheSame`: نفس المنتج؟ يقارن الـ `id`.
- `areContentsTheSame`: هل تغيّرت بياناته؟ يقارن كل الحقول عبر `equals` الخاصة بـ `data class`.

44. `res/layout/fragment_products.xml`

الدور: تصميم شاشة القائمة.

app/src/main/res/layout/fragment_products.xml

26 lines

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent">
6
7      <androidx.recyclerview.widget.RecyclerView
8          android:id="@+id/products_list"
9          android:layout_width="match_parent"
10         android:layout_height="match_parent"
11         android:clipToPadding="false"
12         android:padding="8dp"
13         app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager" />
14
15     <com.google.android.material.progressindicator.CircularProgressIndicator
16         android:id="@+id/progress"
17         android:layout_width="wrap_content"
18         android:layout_height="wrap_content"
19         android:layout_gravity="center"
20         android:indeterminate="true" />
21
22     <include
23         android:id="@+id/error_view"

```

```

24         layout="@layout/view_error"
25         android:visibility="gone" />
26 </FrameLayout>

```

الشرح

- `FrameLayout` : العناصر فوق بعضها، ونُظهر واحداً حسب الحالة: القائمة، أو التحميل، أو الخطأ.
- `app:layoutManager="LinearLayoutManager"` : ترتيب الصفوف عمودياً.
- `clipToPadding="false"` : الصفوف تُرسم داخل الهامش أثناء التمرير، وهذا يعطي شكلاً أجمل.

45. `res/menu/menu_products.xml`

الدور: القائمة العلوية (:) في شاشة المنتجات.

```

app/src/main/res/menu/menu_products.xml 19 lines

1 <?xml version="1.0" encoding="utf-8"?>
2 <menu xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto">
4
5     <item
6         android:id="@+id/action_refresh"
7         android:title="@string/menu_refresh"
8         app:showAsAction="never" />
9
10    <item
11        android:id="@+id/action_concurrency_test"
12        android:title="@string/menu_concurrency_test"
13        app:showAsAction="never" />
14
15    <item
16        android:id="@+id/action_logout"
17        android:title="@string/menu_logout"
18        app:showAsAction="never" />
19 </menu>

```

الشرح

ثلاثة خيارات داخل القائمة المنسدلة (`showAsAction="never"`):

- **Refresh**: إعادة تحميل المنتجات.
- **Run concurrency test**: اختبار التزامن.
- **Log out**: تسجيل الخروج.

46. `ui/products/ProductsFragment.kt`

الدور: شاشة قائمة المنتجات.

```

app/src/main/java/com/example/fakestore/ui/products/ProductsFragment.kt 106 lines

1 package com.example.fakestore.ui.products
2

```

```

3 import android.os.Bundle
4 import android.view.Menu
5 import android.view.MenuInflater
6 import android.view.MenuItem
7 import android.view.View
8 import androidx.core.os.bundleOf
9 import androidx.core.view.MenuProvider
10 import androidx.core.view.isVisible
11 import androidx.fragment.app.Fragment
12 import androidx.fragment.app.viewModels
13 import androidx.lifecycle.Lifecycle
14 import androidx.navigation.fragment.findNavController
15 import com.example.fakestore.R
16 import com.example.fakestore.data.model.Product
17 import com.example.fakestore.databinding.FragmentProductsBinding
18 import com.example.fakestore.ui.common.UiState
19 import com.example.fakestore.ui.common.collectWhenStarted
20 import com.example.fakestore.ui.detail.ProductDetailViewModel
21 import com.google.android.material.snackbar.Snackbar
22 import dagger.hilt.android.AndroidEntryPoint
23
24 @AndroidEntryPoint
25 class ProductsFragment : Fragment(R.layout.fragment_products) {
26
27     private val viewModel: ProductsViewModel by viewModels()
28
29     private var _binding: FragmentProductsBinding? = null
30     private val binding get() = _binding!!
31
32     private val adapter = ProductAdapter { product ->
33         findNavController().navigate(
34             R.id.action_products_to_detail,
35             bundleOf(ProductDetailViewModel.ARG_PRODUCT_ID to product.id),
36         )
37     }
38
39     override fun onCreateView(view: View, savedInstanceState: Bundle?) {
40         super.onCreateView(view, savedInstanceState)
41         _binding = FragmentProductsBinding.bind(view)
42
43         binding.productsList.adapter = adapter
44         binding.errorView.retryButton.setOnClickListener { viewModel.loadProducts() }
45         setUpMenu()
46
47         collectWhenStarted(viewModel.state) { render(it) }
48         collectWhenStarted(viewModel.events) { handle(it) }
49     }
50
51     private fun render(state: UiState<List<Product>>) {
52         binding.progress.isVisible = state is UiState.Loading
53         binding.productsList.isVisible = state is UiState.Success
54         binding.errorView.root.isVisible = state is UiState.Error
55
56         when (state) {
57             is UiState.Success -> adapter.submitList(state.data)
58             is UiState.Error -> binding.errorView.errorMessage.setText(state.messageRes)
59             UiState.Loading -> Unit
60         }
61     }
62
63     private fun handle(event: ProductsEvent) {
64         when (event) {
65             ProductsEvent.LoggedOut -> findNavController().navigate(R.id.action_global_login)
66             is ProductsEvent.ConcurrencyTestFinished -> Snackbar.make(
67                 binding.root,
68                 getString(R.string.concurrency_test_done, event.requestCount),

```

```

69         Snackbar.LENGTH_LONG,
70     ).show()
71     is ProductsEvent.ShowError -> Snackbar.make(
72         binding.root, event.messageRes, Snackbar.LENGTH_LONG,
73     ).show()
74 }
75 }
76
77 private fun setUpMenu() {
78     requireActivity().addMenuProvider(object : MenuProvider {
79         override fun onCreateMenu(menu: Menu, menuInflater: MenuInflater) {
80             menuInflater.inflate(R.menu.menu_products, menu)
81         }
82
83         override fun onMenuItemSelected(menuItem: MenuItem): Boolean = when (menuItem.itemId) {
84             R.id.action_refresh -> {
85                 viewModel.loadProducts()
86                 true
87             }
88             R.id.action_concurrency_test -> {
89                 viewModel.runConcurrencyTest()
90                 true
91             }
92             R.id.action_logout -> {
93                 viewModel.logout()
94                 true
95             }
96             else -> false
97         }
98     }, viewLifecycleOwner, Lifecycle.State.RESUMED)
99 }
100
101 override fun onDestroyView() {
102     super.onDestroyView()
103     binding.productsList.adapter = null // the adapter outlives the view: avoid leaking it
104     _binding = null
105 }
106 }

```

الشرح

adapter: يُنشأ مرة واحدة. عند الضغط على منتج نفتح التفاصيل مع رقمه:

```
bundleOf("productId" to product.id)
```

: render(state)

- **isVisible**: نُظهر عنصراً واحداً فقط حسب الحالة.
- **.adapter.submitList(...)**: Success
- **Error**: نضع نص الخطأ.

: handle(event)

- **LoggedOut**: الانتقال لـ Login مع مسح كل الشاشات.
- **ShowError** / **ConcurrencyTestFinished**: رسالة **Snackbar** أسفل الشاشة.

: setUpMenu()

- **MenuProvider** هو الطريقة الحديثة لإضافة قائمة من Fragment.
- **viewLifecycleOwner, RESUMED**: القائمة تظهر فقط والشاشة نشطة، وتُزال تلقائياً عند مغادرتها.

: onDestroyView

- adapter : productsList.adapter = null
- ثم ._binding = null

47. ui/detail/ProductDetailViewModel.kt

الدور: منطق شاشة التفاصيل.

app/src/main/java/com/example/fakestore/ui/detail/ProductDetailViewModel.kt

53 lines

```
1 package com.example.fakestore.ui.detail
2
3 import androidx.lifecycle.SavedStateHandle
4 import androidx.lifecycle.ViewModel
5 import androidx.lifecycle.ViewModelScope
6 import com.example.fakestore.data.model.Product
7 import com.example.fakestore.data.repository.ProductRepository
8 import com.example.fakestore.ui.common.UiState
9 import com.example.fakestore.ui.common.toMessageRes
10 import dagger.hilt.android.lifecycle.HiltViewModel
11 import kotlinx.coroutines.CancellationException
12 import kotlinx.coroutines.flow.MutableStateFlow
13 import kotlinx.coroutines.flow.StateFlow
14 import kotlinx.coroutines.flow.asStateFlow
15 import kotlinx.coroutines.launch
16 import javax.inject.Inject
17
18 @HiltViewModel
19 class ProductDetailViewModel @Inject constructor(
20     savedStateHandle: SavedStateHandle,
21     private val repository: ProductRepository,
22 ) : ViewModel() {
23
24     // The navigation argument arrives through SavedStateHandle and survives process death.
25     private val productId: Int = checkNotNull(savedStateHandle[ARG_PRODUCT_ID]) {
26         "ProductDetailViewModel requires the '$ARG_PRODUCT_ID' argument"
27     }
28
29     private val _state = MutableStateFlow<UiState<Product>>(UiState.Loading)
30     val state: StateFlow<UiState<Product>> = _state.asStateFlow()
31
32     init {
33         loadProduct()
34     }
35
36     fun loadProduct() {
37         viewModelScope.launch {
38             _state.value = UiState.Loading
39             _state.value = try {
40                 UiState.Success(repository.getProduct(productId))
41             } catch (e: CancellationException) {
42                 throw e
43             } catch (e: Exception) {
44                 UiState.Error(e.toMessageRes())
45             }
46         }
47     }
48
49     companion object {
50         /** Must match the `<argument android:name>` in nav_graph.xml. */
```

```

51     const val ARG_PRODUCT_ID = "productId"
52 }
53 }

```

الشرح

`SavedStateHandle`: يحمل الـ arguments القادمة من Navigation، أي `productId`.

- **ميزته:** يبقى حتى لو أغلق النظام التطبيق في الخلفية لتوفير الذاكرة (**process death**) ثم أعاده المستخدم.
- `checkNotNull(...)`: إذا لم يصل الرقم، فهذا خطأ برمجي، فنفسل فوراً برسالة واضحة.

`: ARG_PRODUCT_ID = "productId"`

- يجب أن يطابق اسم الـ argument في `nav_graph.xml`.
- ثابت واحد يستخدمه الطرفان، فلا يحدث خطأ إملائي.
- `LoadProduct()`: نفس نمط القائمة. وطلب المنتج أيضاً يمر بالـ `Interceptor` تلقائياً.

48. `res/layout/fragment_product_detail.xml`

الدور: تصميم شاشة التفاصيل.

app/src/main/res/layout/fragment_product_detail.xml

77 lines

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent">
5
6      <androidx.core.widget.NestedScrollView
7          android:id="@+id/content"
8          android:layout_width="match_parent"
9          android:layout_height="match_parent"
10         android:visibility="gone">
11
12         <LinearLayout
13             android:layout_width="match_parent"
14             android:layout_height="wrap_content"
15             android:orientation="vertical"
16             android:padding="16dp">
17
18             <ImageView
19                 android:id="@+id/image"
20                 android:layout_width="match_parent"
21                 android:layout_height="260dp"
22                 android:background="@android:color/white"
23                 android:contentDescription="@string/product_image"
24                 android:padding="16dp"
25                 android:scaleType="fitCenter" />
26
27             <TextView
28                 android:id="@+id/title"
29                 android:layout_width="match_parent"
30                 android:layout_height="wrap_content"
31                 android:layout_marginTop="16dp"
32                 android:textAppearance="?attr/textAppearanceTitleLarge" />
33
34             <TextView
35                 android:id="@+id/price"

```

```

36         android:layout_width="wrap_content"
37         android:layout_height="wrap_content"
38         android:layout_marginTop="8dp"
39         android:textAppearance="?attr/textAppearanceHeadlineSmall"
40         android:textColor="?attr/colorPrimary" />
41
42     <TextView
43         android:id="@+id/category"
44         android:layout_width="wrap_content"
45         android:layout_height="wrap_content"
46         android:layout_marginTop="8dp"
47         android:textAppearance="?attr/textAppearanceLabelLarge" />
48
49     <TextView
50         android:id="@+id/rating"
51         android:layout_width="wrap_content"
52         android:layout_height="wrap_content"
53         android:layout_marginTop="4dp"
54         android:textAppearance="?attr/textAppearanceBodyMedium" />
55
56     <TextView
57         android:id="@+id/description"
58         android:layout_width="match_parent"
59         android:layout_height="wrap_content"
60         android:layout_marginTop="16dp"
61         android:lineSpacingMultiplier="1.2"
62         android:textAppearance="?attr/textAppearanceBodyLarge" />
63 </LinearLayout>
64 </androidx.core.widget.NestedScrollView>
65
66 <com.google.android.material.progressindicator.CircularProgressIndicator
67     android:id="@+id/progress"
68     android:layout_width="wrap_content"
69     android:layout_height="wrap_content"
70     android:layout_gravity="center"
71     android:indeterminate="true" />
72
73 <include
74     android:id="@+id/error_view"
75     layout="@layout/view_error"
76     android:visibility="gone" />
77 </FrameLayout>

```

الشرح

- `NestedScrollView` (معرفه `content`): المحتوى قابل للتمرير، لأن الوصف قد يكون طويلاً. مخفي حتى تصل البيانات.
- المحتوى بالترتيب: صورة كبيرة 260dp، ثم العنوان، والسعر (بلون التيم الأساسي)، والتصنيف، والتقييم، والوصف.
- `lineSpacingMultiplier="1.2"`: مسافة أكبر بين الأسطر لتسهيل قراءة الوصف.

49. `ui/detail/ProductDetailFragment.kt`

الدور: شاشة التفاصيل.

app/src/main/java/com/example/fakestore/ui/detail/ProductDetailFragment.kt

58 lines

```

1 package com.example.fakestore.ui.detail
2
3 import android.os.Bundle
4 import android.view.View

```

```

5 import androidx.core.view.isVisible
6 import androidx.fragment.app.Fragment
7 import androidx.fragment.app.viewModels
8 import coil.load
9 import com.example.fakestore.R
10 import com.example.fakestore.data.model.Product
11 import com.example.fakestore.databinding.FragmentProductDetailBinding
12 import com.example.fakestore.ui.common.UiState
13 import com.example.fakestore.ui.common.collectWhenStarted
14 import com.example.fakestore.ui.common.formatPrice
15 import dagger.hilt.android.AndroidEntryPoint
16
17 @AndroidEntryPoint
18 class ProductDetailFragment : Fragment(R.layout.fragment_product_detail) {
19
20     private val viewModel: ProductDetailViewModel by viewModels()
21
22     private var _binding: FragmentProductDetailBinding? = null
23     private val binding get() = _binding!!
24
25     override fun onCreateView(view: View, savedInstanceState: Bundle?) {
26         super.onCreateView(view, savedInstanceState)
27         _binding = FragmentProductDetailBinding.bind(view)
28
29         binding.errorView.retryButton.setOnClickListener { viewModel.loadProduct() }
30         collectWhenStarted(viewModel.state) { render(it) }
31     }
32
33     private fun render(state: UiState<Product>) {
34         binding.progress.isVisible = state is UiState.Loading
35         binding.content.isVisible = state is UiState.Success
36         binding.errorView.root.isVisible = state is UiState.Error
37
38         when (state) {
39             is UiState.Success -> showProduct(state.data)
40             is UiState.Error -> binding.errorView.errorMessage.setText(state.messageRes)
41             UiState.Loading -> Unit
42         }
43     }
44
45     private fun showProduct(product: Product) {
46         binding.image.load(product.imageUrl) { crossfade(true) }
47         binding.title.text = product.title
48         binding.price.text = product.price.formatPrice()
49         binding.category.text = product.category
50         binding.rating.text = getString(R.string.product_rating, product.rating, product.ratingCount)
51         binding.description.text = product.description
52     }
53
54     override fun onDestroyView() {
55         super.onDestroyView()
56         _binding = null
57     }
58 }

```

الشرح

- نفس نمط شاشة القائمة: binding، ومراقبة الحالة، و render.
- showProduct(): يملأ العناصر. التقييم يُبنى من مورد نصي بقيمتين:

```
getString(R.string.product_rating, rating, count) → "★ 4.1 (259 reviews)"
```

- سهم الرجوع في الشريط العلوي يعمل تلقائياً، لأن التفاصيل ليست شاشة رئيسية في AppBarConfiguration.

الجزء السابع: الاختبارات (Unit Tests)

مكان الاختبارات: المجلد `app/src/test/`. تعمل على الكمبيوتر مباشرة (JVM)، بلا هاتف أو محاكي. طريقة التشغيل: الأمر `./gradlew test`، أو بزر ▶ بجانب أي اختبار في Android Studio.

النتيجة: 15 اختباراً، كلها ناجحة.

- شُغِلت 5 مرات متتالية للتأكد من أن اختبارات التزامن ثابتة وغير عشوائية.
- وأُثبت أنها تكشف الخطأ فعلاً: عند حذف التحقق الثاني داخل القفل، فشل اختباران:
 - 10 طلبات متزامنة أنتجت 10 طلبات login بدل 1.
 - 5 طلبات Retrofit متوازية أنتجت 5 طلبات login بدل 1.

50. `test/.../TestDoubles.kt`

الدور: بدائل وهمية (Test Doubles) للاختبار.

app/src/test/java/com/example/fakestore/data/auth/TestDoubles.kt

106 lines

```
1 package com.example.fakestore.data.auth
2
3 import com.example.fakestore.data.remote.AuthApi
4 import com.example.fakestore.data.remote.ProductApi
5 import okhttp3.OkHttpClient
6 import okhttp3.mockwebserver.Dispatcher
7 import okhttp3.mockwebserver.MockResponse
8 import okhttp3.mockwebserver.MockWebServer
9 import okhttp3.mockwebserver.RecordedRequest
10 import retrofit2.Retrofit
11 import retrofit2.converter.gson.GsonConverterFactory
12 import java.util.concurrent.atomic.AtomicInteger
13
14 /** Clock whose time only moves when the test says so. */
15 class FakeClock(var nowMillis: Long = 1_000_000L) : Clock {
16     override fun nowMillis(): Long = nowMillis
17     fun advanceBy(millis: Long) {
18         nowMillis += millis
19     }
20 }
21
22 /** Thread-safe in-memory replacement for the encrypted storage. */
23 class InMemoryTokenStorage(initial: AuthSession? = null) : TokenStorage {
24     @Volatile
25     var session: AuthSession? = initial
26
27     override fun read(): AuthSession? = session
28     override fun save(session: AuthSession) {
29         this.session = session
30     }
31     override fun clear() {
32         session = null
33     }
34 }
35
36 /**
37  * Local HTTP server that imitates FakeStore.
38  * - `POST /auth/login` returns `token-1`, `token-2`, ... (or [loginResponseCode] if not 200).
```

```

39  * - `GET /products/...` returns [productsResponseCode] with a small JSON body.
40  */
41  class FakeStoreServer {
42      val server = MockWebServer()
43      val loginCount = AtomicInteger()
44      val productRequests = AtomicInteger()
45
46      @Volatile var loginResponseCode = 200
47      @Volatile var loginDelayMillis = 0L
48      @Volatile var productsResponseCode = 200
49
50      private val seenAuthHeaders = java.util.concurrent.ConcurrentLinkedQueue<String?>()
51      val productAuthHeaders: List<String?> get() = seenAuthHeaders.toList()
52
53      init {
54          server.dispatcher = object : Dispatcher() {
55              override fun dispatch(request: RecordedRequest): MockResponse {
56                  val path = request.path.orEmpty()
57                  return when {
58                      path.startsWith("/auth/login") -> {
59                          val n = loginCount.incrementAndGet()
60                          if (loginDelayMillis > 0) Thread.sleep(loginDelayMillis)
61                          if (loginResponseCode == 200) {
62                              MockResponse().setBody("""{"token":"token-$n"}""")
63                          } else {
64                              MockResponse().setResponseCode(loginResponseCode)
65                                  .setBody("username or password is incorrect")
66                          }
67                      }
68                      path.startsWith("/products") -> {
69                          productRequests.incrementAndGet()
70                          seenAuthHeaders.add(request.getHeader(AuthHeaders.AUTHORIZATION))
71                          MockResponse().setResponseCode(productsResponseCode).setBody(
72                              if (path == "/products") "[]"
73                              else """
74{id":1,"title":"Bag","price":10.5,"description":"d","category":"c","image":"i","rating":
75{"rate":4.1,"count":7}}"""
76                      )
77                      else -> MockResponse().setResponseCode(404)
78                  }
79              }
80          }
81          server.start()
82      }
83
84      private fun retrofit(client: OkHttpClient): Retrofit = Retrofit.Builder()
85          .baseUrl(server.url("/"))
86          .client(client)
87          .addConverterFactory(GsonConverterFactory.create())
88          .build()
89
90      /** Same wiring as NetworkModule: login on a plain client, products on an authenticated one. */
91      fun authApi(): AuthApi = retrofit(OkHttpClient()).create(AuthApi::class.java)
92
93      fun productApi(authManager: AuthManager): ProductApi {
94          val client = OkHttpClient.Builder()
95              .addInterceptor(AuthInterceptor(authManager))
96              .authenticator(TokenAuthenticator(authManager))
97              .build()
98          return retrofit(client).create(ProductApi::class.java)
99      }
100
101      fun shutdown() = server.shutdown()
102  }

```

```

103 val TEST_CREDENTIALS = Credentials("mor_2314", "83r5^_")
104
105 fun sessionSavedAt(clock: FakeClock, token: String = "token-0") =
106     AuthSession(token, clock.nowMillis, TEST_CREDENTIALS)

```

الشرح

FakeClock: ساعة لا تتحرك إلا بأمر منا.

- `advanceBy(61_000)` = "مرت 61 ثانية" فوراً.

InMemoryTokenStorage: تخزين في الذاكرة بدل التشفير.

- `@Volatile`: يضمن أن كل ال threads ترى آخر قيمة مكتوبة.

FakeStoreServer: يستخدم **MockWebServer**، وهو سيرفر HTTP حقيقي يعمل على الكمبيوتر محلياً ويقلد **FakeStore**:

- `/auth/login`: يرد `token-1` ثم `token-2` ... وبعد كم مرة استدعي (`loginCount`).

- `/products`: يرد بمنتجات، ويسجل ال header الذي وصله (`productAuthHeaders`).

- `loginDelayMillis`: يؤخر رد `login`، ليبقى التجديد "جارياً" بينما تصل الطلبات الأخرى. **ضروري لاختبار التزامن بصدق.**

- `loginResponseCode` / `productsResponseCode`: لمحاكاة الرفض (401).

productApi(authManager): يبني عميلاً محمياً بنفس طريقة **NetworkModule**، فنختبر التركيب الحقيقي نفسه.

51. test/.../AuthManagerTest.kt

الدور: اختبار منطق **AuthManager** مباشرة.

app/src/test/java/com/example/fakestore/data/auth/AuthManagerTest.kt

166 lines

```

1 package com.example.fakestore.data.auth
2
3 import kotlinx.coroutines.ExperimentalCoroutinesApi
4 import kotlinx.coroutines.flow.toList
5 import kotlinx.coroutines.launch
6 import kotlinx.coroutines.test.UnconfinedTestDispatcher
7 import kotlinx.coroutines.test.runTest
8 import org.junit.After
9 import org.junit.Assert.assertEquals
10 import org.junit.Assert.assertNotNull
11 import org.junit.Assert.assertNull
12 import org.junit.Assert.assertTrue
13 import org.junit.Assert.fail
14 import org.junit.Test
15 import java.io.IOException
16 import java.util.concurrent.CountDownLatch
17 import java.util.concurrent.Executors
18 import java.util.concurrent.TimeUnit
19
20 @OptIn(ExperimentalCoroutinesApi::class)
21 class AuthManagerTest {
22
23     private val server = FakeStoreServer()
24     private val clock = FakeClock()
25     private val storage = InMemoryTokenStorage()
26     private val authManager = AuthManager(server.authApi(), storage, clock)
27
28     @After

```

```

29     fun tearDown() = server.shutdown()
30
31     @Test
32     fun `login stores token and tokenSavedAt`() = runTest {
33         authManager.login("mor_2314", "83r5^_")
34
35         val session = storage.session!!
36         assertEquals("token-1", session.token)
37         assertEquals(clock.nowMillis, session.savedAtMillis)
38         assertTrue(authManager.isLoggedIn())
39     }
40
41     @Test
42     fun `login with wrong password throws InvalidCredentialsException`() = runTest {
43         server.loginResponseCode = 401
44         try {
45             authManager.login("mor_2314", "wrong")
46             fail("Expected InvalidCredentialsException")
47         } catch (e: InvalidCredentialsException) {
48             assertEquals(401, e.httpCode)
49         }
50         assertNull(storage.session)
51     }
52
53     @Test
54     fun `valid token is returned without any network call`() {
55         storage.session = sessionSavedAt(clock)
56         clock.advanceBy(59_000)
57
58         assertEquals("token-0", authManager.getValidToken())
59         assertEquals(0, server.loginCount.get())
60     }
61
62     @Test
63     fun `token is treated as expired after exactly 60 seconds and refreshed`() {
64         storage.session = sessionSavedAt(clock)
65         clock.advanceBy(AuthManager.TOKEN_LIFETIME_MS)
66
67         assertEquals("token-1", authManager.getValidToken())
68         assertEquals(1, server.loginCount.get())
69         assertEquals(clock.nowMillis, storage.session!!.savedAtMillis)
70     }
71
72     @Test
73     fun `clock moving backwards is treated as expired`() {
74         storage.session = sessionSavedAt(clock)
75         clock.advanceBy(-1)
76
77         assertEquals("token-1", authManager.getValidToken())
78     }
79
80     @Test
81     fun `concurrent callers with an expired token trigger exactly one refresh`() {
82         storage.session = sessionSavedAt(clock)
83         clock.advanceBy(61_000)
84         server.loginDelayMillis = 300 // keep the refresh in flight while others arrive
85
86         val threads = 10
87         val pool = Executors.newFixedThreadPool(threads)
88         val start = CountDownLatch(1)
89         val results = (1..threads).map {
90             pool.submit<String> {
91                 start.await()
92                 authManager.getValidToken()
93             }
94         }

```

```

95     start.countDown()
96     val tokens = results.map { it.get(10, TimeUnit.SECONDS) }
97     pool.shutdown()
98
99     assertEquals(1, server.loginCount.get())
100    assertTrue(tokens.all { it == "token-1" })
101 }
102
103 @Test
104 fun `rejected refresh forces logout and emits SessionExpired`() = runTest {
105     val events = mutableListOf<AuthEvent>()
106     backgroundScope.launch(UnconfinedTestDispatcher(testScheduler)) {
107         authManager.events.toList(events)
108     }
109     storage.session = sessionSavedAt(clock)
110     clock.advanceBy(61_000)
111     server.loginResponseCode = 401
112
113     try {
114         authManager.getValidToken()
115         fail("Expected SessionExpiredException")
116     } catch (expected: SessionExpiredException) {
117     }
118
119     assertNull(storage.session)
120     assertEquals(listOf<AuthEvent>(AuthEvent.SessionExpired), events)
121     assertEquals(1, server.loginCount.get()) // no retry loop
122 }
123
124 @Test
125 fun `network failure during refresh keeps the session`() {
126     storage.session = sessionSavedAt(clock)
127     clock.advanceBy(61_000)
128     server.shutdown()
129
130     try {
131         authManager.getValidToken()
132         fail("Expected IOException")
133     } catch (e: IOException) {
134         assertTrue(e !is SessionExpiredException)
135     }
136     assertNotNull(storage.session)
137 }
138
139 @Test
140 fun `no session means SessionExpiredException without network`() {
141     try {
142         authManager.getValidToken()
143         fail("Expected SessionExpiredException")
144     } catch (expected: SessionExpiredException) {
145     }
146     assertEquals(0, server.loginCount.get())
147 }
148
149 @Test
150 fun `logout during a refresh does not bring the session back`() {
151     storage.session = sessionSavedAt(clock)
152     clock.advanceBy(61_000)
153     server.loginDelayMillis = 300
154
155     val pool = Executors.newSingleThreadExecutor()
156     val refresh = pool.submit<Result<String>> { runCatching { authManager.getValidToken() } }
157     Thread.sleep(100) // refresh is now waiting for the server
158     authManager.logout()
159
160     val result = refresh.get(5, TimeUnit.SECONDS)

```

```

161     pool.shutdown()
162
163     assertTrue(result.exceptionOrNull() is SessionExpiredException)
164     assertNull(storage.session)
165 }
166 }

```

الشرح

الاختبار	ماذا يثبت
login stores token and tokenSavedAt	الدخول يحفظ الـ token مع وقت الحفظ
login with wrong password...	كلمة مرور خاطئة تعطي InvalidCredentialsException ، ولا يُحفظ شيء
valid token is returned without any network call	عند 59 ثانية: لا يوجد أي طلب شبكة
token is treated as expired after exactly 60 seconds	عند 60 ثانية بالضبط: تجديد، و tokenSavedAt جديد
clock moving backwards...	تغيير ساعة الجهاز للخلف يعني اعتبار الـ token منتهياً
concurrent callers ... exactly one refresh ★	10 threads معاً تنتج login واحداً، وكلها تحصل على نفس الـ token
rejected refresh forces logout...	رفض التجديد يعني مسح الجلسة، وإرسال الحدث، ومحاولة واحدة فقط
network failure during refresh keeps the session	انقطاع الإنترنت لا يُخرج المستخدم
no session means SessionExpiredException...	بلا جلسة: خطأ دون أي طلب
logout during a refresh...	Logout أثناء التجديد لا يعيد الجلسة

كيف يعمل اختبار التزامن؟

1. Executors.newFixedThreadPool(10) : عشرة threads حقيقية.
 2. CountdownLatch(1) : "خط البداية". كل الـ threads تنتظر عنده.
 3. start.countDown() : تنطلق العشرة في نفس اللحظة.
 4. التحقق: loginCount == 1 ، وكل النتائج تساوي token-1 .
- runTest : يشغل كود coroutines في الاختبار. backgroundScope : يستمع للأحداث في الخلفية، ويُلغى تلقائياً في نهاية الاختبار.

52. test/.../AuthInterceptorTest.kt

الدور: اختبار متكامل (integration test): Retrofit + OkHttp + Interceptor + AuthManager معاً، أمام سيرفر محلي.

```

1  package com.example.fakestore.data.auth
2
3  import com.example.fakestore.data.remote.ProductApi
4  import kotlinx.coroutines.async
5  import kotlinx.coroutines.awaitAll
6  import kotlinx.coroutines.runBlocking
7  import org.junit.After
8  import org.junit.Assert.assertEquals
9  import org.junit.Assert.assertTrue
10 import org.junit.Assert.fail
11 import org.junit.Test
12
13 /** End-to-end: Retrofit + OkHttp + AuthInterceptor + AuthManager against a local server. */
14 class AuthInterceptorTest {
15
16     private val server = FakeStoreServer()
17     private val clock = FakeClock()
18     private val storage = InMemoryTokenStorage()
19     private val authManager = AuthManager(server.authApi(), storage, clock)
20     private val productApi: ProductApi = server.productApi(authManager)
21
22     @After
23     fun tearDown() = server.shutdown()
24
25     @Test
26     fun `every product request carries the bearer token`() = runBlocking {
27         storage.session = sessionSavedAt(clock, token = "abc")
28
29         productApi.getProducts()
30         productApi.getProduct(1)
31
32         assertEquals(listOf("Bearer abc", "Bearer abc"), server.productAuthHeaders)
33         assertEquals(0, server.loginCount.get())
34     }
35
36     @Test
37     fun `expired token is refreshed before the request is sent`() = runBlocking {
38         storage.session = sessionSavedAt(clock, token = "old")
39         clock.advanceBy(61_000)
40
41         productApi.getProducts()
42
43         assertEquals(1, server.loginCount.get())
44         assertEquals(listOf("Bearer token-1"), server.productAuthHeaders) // never "Bearer old"
45     }
46
47     @Test
48     fun `parallel requests with an expired token share one refresh`() = runBlocking {
49         storage.session = sessionSavedAt(clock, token = "old")
50         clock.advanceBy(61_000)
51         server.loginDelayMillis = 300
52
53         (1..5).map { async(kotlinx.coroutines.Dispatchers.IO) { productApi.getProduct(it) } }.awaitAll()
54
55         assertEquals(1, server.loginCount.get())
56         assertEquals(5, server.productRequests.get())
57         assertTrue(server.productAuthHeaders.all { it == "Bearer token-1" })
58     }
59
60     @Test
61     fun `without a session the request fails and never reaches the server`() = runBlocking {
62         try {
63             productApi.getProducts()

```

```

64         fail("Expected SessionExpiredException")
65     } catch (expected: SessionExpiredException) {
66     }
67     assertEquals(0, server.productRequests.get())
68 }
69 }

```

الشرح

الاختبار	ماذا يثبت
every product request carries the bearer token	كل طلب منتجات وصل للسيرفر ومعه Bearer abc
expired token is refreshed before the request is sent	السيرفر لم يستقبل أبداً Bearer old المنتهي، بل Bearer token-1 الجديد مباشرة
parallel requests ... share one refresh ★	5 طلبات Retrofit متوازية تنتج login واحداً، و5 طلبات منتجات، كلها بال token الجديد
without a session the request fails...	بلا جلسة: الطلب لا يصل للسيرفر أصلاً

.53 test/.../TokenAuthenticatorTest.kt

الدور: إثبات عدم وجود حلقة لا نهائية عند رد 401.

app/src/test/java/com/example/fakestore/data/auth/TokenAuthenticatorTest.kt

38 lines

```

1  package com.example.fakestore.data.auth
2
3  import kotlinx.coroutines.runBlocking
4  import org.junit.After
5  import org.junit.Assert.assertEquals
6  import org.junit.Assert.assertNull
7  import org.junit.Test
8  import retrofit2.HttpException
9
10 /** The 401 safety net: refresh + retry once, never loop. */
11 class TokenAuthenticatorTest {
12
13     private val server = FakeStoreServer()
14     private val clock = FakeClock()
15     private val storage = InMemoryTokenStorage()
16     private val authManager = AuthManager(server.authApi(), storage, clock)
17     private val productApi = server.productApi(authManager)
18
19     @After
20     fun tearDown() = server.shutdown()
21
22     @Test
23     fun `server keeps answering 401 - one retry then forced logout`() = runBlocking {
24         storage.session = sessionSavedAt(clock, token = "rejected")
25         server.productsResponseCode = 401
26
27         try {
28             productApi.getProducts()
29         } catch (e: HttpException) {
30             assertEquals(401, e.code())
31         }
32     }
33 }

```

```

31     }
32
33     assertEquals(2, server.productRequests.get()) // original + exactly one retry
34     assertEquals(1, server.loginCount.get())
35     assertEquals(listOf("Bearer rejected", "Bearer token-1"), server.productAuthHeaders)
36     assertNull(storage.session)
37 }
38 }

```

الشرح

السيناريو: السيرفر يرد **401 دائماً**.

النتيجة المثبتة:

- طلبان فقط: الأصلي + إعادة واحدة.
 - طلب login واحد.
 - الجلسة مُسحت (خروج إجباري).
 - ال headers: الأول `Bearer rejected` ، والثاني `Bearer token-1`.
- بدون فحص `priorResponse` كانت المحاولات ستتكرر حتى حد OkHttp الداخلي (20 محاولة).

الجزء الثامن: الصورة الكاملة

54. رحلة طلب كاملة عبر الملفات

السيناريو: المستخدم سجّل دخوله قبل 70 ثانية، ثم ضغط على منتج.

#	أين	ماذا يحدث
1	ProductAdapter	الضغط يستدعي <code>navigate(action_products_to_detail, productId=3)</code>
2	nav_graph.xml	يفتح <code>ProductDetailFragment</code>
3	ProductDetailViewModel	يقرأ <code>productId</code> من <code>SavedStateHandle</code> ، ثم <code>LoadProduct()</code> ، والحالة <code>Loading</code>
4	ProductRepositoryImpl	<code>api.getProduct(3)</code>
5	ProductApi (Retrofit)	يبنى طلب <code>GET /products/3</code> على العميل <code>@AuthClient</code>
6	AuthInterceptor	<code>authManager.isValidToken()</code>
7	AuthManager	المسار السريع: عمر الـ 70 token ثانية، منتج
8	AuthManager	يدخل <code>refreshLock</code> ويتحقق مرة ثانية: ما زال منتهياً
9	AuthManager.refresh	<code>requestNewSession</code> يرسل <code>AuthApi.login</code> على العميل <code>@PlainClient</code> ، بلا Interceptor
10	EncryptedTokenStorage	يحفظ الـ token الجديد مشفراً مع الآن <code>tokenSavedAt =</code>
11	AuthInterceptor	يضيف <جديد> <code>Authorization: Bearer</code> ويرسل الطلب
12	Gson	يحوّل JSON إلى <code>ProductDto</code> ، ثم <code>toDomain()</code> إلى <code>Product</code>
13	ProductDetailViewModel	الحالة <code>Success(product)</code>
14	ProductDetailFragment	<code>render</code> ثم <code>showProduct</code> ، فتظهر التفاصيل

ولو رفض السيرفر بيانات الدخول في الخطوة 9:

1. `forceLogout()`: يسمح الجلسة ويرسل `SessionExpired`.
2. يُرمى `SessionExpiredException` (وهو `IOException`)، فتعرض الشاشة "انتهت الجلسة".
3. `MainActivity` تستقبل الحدث، فتعرض Toast وتنتقل لـ `Login` مع مسح كل الشاشات.

55. كيف تعرض المشروع للمدير (Demo)

1. شغل التطبيق، وافتح نافذة **Logcat** في Android Studio، واكتب في البحث: `okhttp`
2. سجّل الدخول بـ `mor_2314` وكلمة المرور `83r5^_`. سيظهر: `POST https://fakestoreapi.com/auth/login` -->
3. تصفّح المنتجات خلال أول دقيقة: تظهر طلبات `GET /products` فقط، **بلا login**.

4. انتظر أكثر من 60 ثانية، ثم من القائمة (:) اختر **Run concurrency test**. سيظهر: `POST /auth/login` - **مرة واحدة فقط**. - ثم 5 طلبات `GET /products/1..5`.
5. اختر **Log out**: تعود لشاشة Login، وزر الرجوع يغلق التطبيق.
6. افصل الإنترنت ثم اختر **Refresh**: رسالة "No internet connection" مع زر Retry، ولا يحدث crash.
7. شغل الاختبارات بـ `./gradlew test`، واعرض نتيجة الـ 15 اختباراً.

56. تقابل المتطلبات مع الكود

المتطلب في ملف المهمة	الملف	الكود
Username & password inputs, Login button	<code>fragment_login.xml</code>	<code>password_input</code> , <code>username_input</code> <code>login_button</code>
Loading & error states	<code>LoginFragment</code> / <code>LoginViewModel</code>	<code>render()</code> , <code>LoginUiState</code>
Securely store the token	<code>EncryptedTokenStorage</code>	AES-256 + Android Keystore
Authorization Interceptor	<code>AuthInterceptor</code>	<code>.header("Authorization", "Bearer ...")</code>
Token validity: 60 seconds	<code>AuthManager</code>	<code>TOKEN_LIFETIME_MS = 60_000L</code>
Store tokenSavedAt	<code>AuthSession</code>	<code>savedAtMillis</code>
Treat token as expired after 60 s	<code>AuthManager</code>	<code>isExpired()</code>
Do not send requests with an expired token	<code>AuthManager</code>	<code>getValidToken()</code> قبل كل طلب
Trigger refresh flow automatically	<code>AuthManager</code>	<code>refresh()</code>
Only one refresh request must run	<code>AuthManager</code>	<code>synchronized(refreshLock)</code> + التحقق الثاني
Other requests wait and resume	<code>AuthManager</code>	الانتظار على القفل ثم المتابعة بال token الجديد
Simulate refresh by re-calling login	<code>AuthManager</code>	→ <code>requestNewSession()</code> <code>AuthApi.login</code>
Encapsulate inside AuthManager	<code>AuthManager</code>	كل المنطق في كلاس واحد
Avoid infinite retry loops	<code>AuthManager</code> / <code>NetworkModule</code> <code>TokenAuthenticator</code>	عميل منفصل + محاولة واحدة + <code>priorResponse</code>
If refresh fails → force logout	<code>MainActivity</code> / <code>AuthManager</code>	<code>SessionExpired</code> → <code>forceLogout()</code>

المتطلب في ملف المهمة	الملف	الكود
Products list (RecyclerView)	<code>ProductAdapter</code> / <code>fragment_products.xml</code>	<code>DiffUtil</code> + <code>ListAdapter</code>
Product details screen	<code>ProductDetailFragment</code>	
Logout: clear token, go to Login	<code>nav_graph.xml</code> / <code>ProductsViewModel</code>	<code>action_global_login</code> + <code>logout()</code>
Architecture: MVVM	كل الشاشات	Fragment + ViewModel + Repository
Networking: Retrofit + OkHttp	<code>NetworkModule</code>	